Post-Training State-Space Foundation Models for High-Frequency Trading



Alexandre Bismuth

St Anne's College

University of Oxford

A thesis submitted for the degree of

Master of Science in Advanced Computer Science

Trinity Term, 2026

Supervised by Professor Ani Calinescu and Professor Jakob Foerster

Abstract

Autoregressive foundation models offer state-of-the-art capabilities to simulate the market by generating the message flow of limit order books. Nonetheless, their inner workings are not yet understood, obscuring research directions and limiting integration in real-world trading environments. Moreover, their cross-entropy pre-training objective cannot target desired market properties and lets sampling errors compound over long sequence generations. This study thus asks what these models learn about market microstructure, and introduces a post-training refinement stage that scores full sequences rather than single messages. Probing and steering the recurrent state of a Mamba-3 state-space model, we discover a general representation of the market as a flow ($R^2 \approx 0.6$ for order-flow imbalance, far above controls) that can be steered along its encoded features. Training an adversarial critic on messages executed through the non-differentiable matching engine and updating the pre-trained generator with hardware-aware evolutionary search, we improve LOB-Bench realism by 17.6% with three orders of magnitude less data and $< 10\%$ of the compute budget. Neither a selection-matched null nor continued pre-training at matched compute reproduces the gain, so post-training is a stage that more pre-training does not replace. Error accumulation over 500 messages also falls from 3.1× to 2.3×, while the pre-trained representation and directional accuracy stay intact.

Acknowledgements

I would like to thank my supervisor, Professor Ani Calinescu, for her guidance, her dedication at any hour, and her continued written and oral feedback throughout this project. I am also grateful to Professor Jakob Foerster for his invaluable feedback and for the weekly discussions that shaped this work. Together, they gave me the opportunity to learn and grow in the welcoming and active research environment of the British Open-ended Learning and Discovery Lab (BOLD). Then, I want to thank Kang Li for inviting me to the machine learning for finance team and for the many hours spent studying papers and reviewing results. Furthermore, my thanks also go to Valentin Möhl for working alongside me on the Chapter 5 experiments of this thesis, and to Aramis Marti-Shahandeh for his detailed comments. Lastly, I would like to thank everyone else at BOLD for their warmth, creativity and open-mindedness, which have made my MSc thesis experience a great one.

The author acknowledges the use of resources provided by the Isambard-AI National AI Research Resource (AIRR). Isambard-AI is operated by the University of Bristol and is funded by the UK Government's Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

Contents

Glossary **vi**

Abbreviations **ix**

1 Introduction **1**

1.1 Contributions 3

1.2 Scope 4

1.3 Thesis Outline 4

2 Background **6**

2.1 Market Microstructure 7

2.2 Pre-Training on Limit Order Books 8

2.3 Evaluating Limit Order Book Realism 12

2.4 Mechanistic Interpretability 16

2.5 Generative Adversarial Networks 19

2.6 Post-Training Optimisations 21

3 Related Work **26**

3.1 Feature Engineering for Market Predictions 27

3.2 Generative Models for Market Simulation 28

3.3 Mechanistic Interpretability of State-Space Models 30

3.4 Post-Training and Zero-Order Optimisation 31

3.5 Positioning 33

4 Methodology **34**

4.1 Dataset 35

4.2 Tokenisation 37

4.3 Pre-Trained Generators 38

4.4 Inference Pipeline 41

4.5 Evaluation Protocol 42
4.6 Training Environment 43

5 Understanding Foundation Models via Mechanistic Interpretability 45
5.1 Mechanistic Interpretability Pipeline 46
5.2 Encoding of Market Statistics 50
5.3 Forward Predictions 55
5.4 Learning Dynamics 59
5.5 Summary 61

6 Adversarial Post-Training with Evolutionary Search 62
6.1 Adversarial Pipeline 63
6.2 Results 67
6.3 Training Dynamics 68
6.4 Compute Efficiency 69
6.5 Critic and Optimiser Ablations 71
6.6 What Post-Training Changes 72
6.7 Summary 79

7 Discussion 81
7.1 Current Understanding of HFT Foundation Models 82
7.2 Integration into High-Frequency Trading 85

8 Conclusion 88
8.1 Summary of Findings 88
8.2 Limitations 90
8.3 Future Work 91

Appendices

A Low-Rank Evolution Strategies 95

B LOB-Bench Scoring Functions 97

C Hyperparameters 99

References 101

Glossary

Anchor

The frozen pre-trained model that post-training starts from, and the reference against which its results are measured.

Autoregressive training

Fitting a model to predict the next element from those preceding it, by minimising the cross-entropy given the true context. Generation proceeds one element at a time, each sample being fed back as the next input.

Bid–ask spread

The distance between the best ask and the best bid. It is the immediate round-trip cost of trading and a standard measure of market quality.

Block bootstrap

Resampling contiguous blocks of observations, such as whole evaluation windows or whole trading days, rather than individual points, so that dependence within a block is preserved.

Exposure bias

The mismatch between training, where the model always conditions on real history, and generation, where it conditions on its own output. Never having seen its own mistakes, the model drifts into states it has not learned to continue.

High-frequency trading

Automated trading at sub-millisecond latencies, where queue position and message timing determine profitability.

IsoFLOP

Matched on total floating-point operations. An IsoFLOP control spends the same compute on continued pre-training as the run it is compared against.

Level-1/2/3 data

The three granularities of market data: level 1 gives the touch, level 2 the aggregated volume at each price level, and level 3 the individual messages. This work uses level-3 data.

Limit order book

The exchange's record of every outstanding buy and sell limit order, organised by price level. It is obtained by applying the matching rules to the stream of arriving messages.

Limit order

An order to buy or sell a stated quantity at a price no worse than a stated limit. If unexecuted immediately, it rests in the book as visible liquidity until executed or cancelled.

Liquidity

The capacity of a market to absorb a trade without a large price move. Resting liquidity is the volume waiting in the book; consuming it moves the price.

Market impact

The price move caused by one's own trading. It is the dominant cost of executing a large order.

Matching engine

The exchange component that applies the matching rules to arriving messages and decides which orders trade against which. It is deterministic but discrete, so no gradient passes through it.

Message stream

The time-ordered sequence of order submissions, cancellations and executions that an exchange processes. It is a lossless record of market activity, from which the book at any instant can be reconstructed.

Mid-price

The average of the best bid and the best ask, and the usual reference price for measuring returns and directional moves.

Order flow

The arriving message stream viewed as a signed process, netting buying pressure against selling pressure.

Post-training

A stage after pre-training that optimises a model against a criterion computed on its own generations rather than on next-token likelihood.

Pre-training

The initial stage that fits a model to a large corpus by next-token likelihood alone, with no objective specific to how it will later be used. It yields the general-purpose model that subsequent stages start from.

Price–time priority

The matching rule of equity exchanges: better prices first, and price-matched orders execute in order of arrival. It is what makes queue position valuable.

Price level

A price in the book, together with the queue of resting orders. Levels count outwards from the touch, so L10 denotes the ten best levels on each side.

Random-weights twin

An untrained network with the same architecture and initialisation as the model under study, probed identically. It separates what training has built from what the architecture supplies on its own.

Residual stream

The per-token vector that Transformer layers read from and write to, and the usual site of interpretability work. In a state-space model it persists only within a token position.

Reward hacking

Raising a learned reward by exploiting the model rather than by improving the property it was meant to measure. Against an online critic it appears as a point past which held-out quality decays while the reward keeps rising.

State-space model

A sequence architecture that carries information in a fixed-size recurrent state updated at every token, rather than by attending over all previous tokens. Inference is linear in sequence length.

Teacher forcing

Forcing each prediction to be conditioned on the real history rather than on the model's own samples. Likelihood training requires it by definition, but it leaves the model untrained on its own output, the root of exposure bias.

Tick

The increment on which prices are quoted. The exchange tick size for US equities above one dollar is one cent; the data used here record prices on the finer grid of 10^{-4} USD.

Touch

The best bid and the best ask, together known as the quotes. Also called the top of the book.

Abbreviations

CI	Confidence Interval
CPT	Continued Pre-Training
CV	Cross-Validation
EGGROLL	Evolution Guided GeneRal Optimisation via Low-rank Learning
ES	Evolution Strategies
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
GRPO	Group Relative Policy Optimization
HFT	High-Frequency Trading
KL	Kullback–Leibler
LLM	Large Language Model
LOB	Limit Order Book
LOBSTER	Limit Order Book Reconstruction System
LoRA	Low-Rank Adaptation
OFI	Order-Flow Imbalance
OOD	Out-Of-Distribution
PCA	Principal Component Analysis
PPO	Proximal Policy Optimization
SAE	Sparse AutoEncoder
SSD	State-Space Duality
SSM	State-Space Model
TCN	Temporal Convolutional Network

1

Introduction

Recent work has established foundation models for High-Frequency Trading (HFT) as the state-of-the-art for market simulation through autoregressive training on Limit Order Book (LOB) messages [1–3]. These models are pre-trained with cross-entropy, maximising next-token likelihood over historical message streams and thus implicitly matching the market's conditional next-message distribution. Electronic markets are well-suited to this methodology: every submission, cancellation and execution is a time-stamped message, feeds such as Limit Order Book Reconstruction System (LOBSTER) [4] preserve the stream losslessly, and the exchange's matching engine folds it deterministically into the book. Such models aim to recreate a synthetic market that reacts accurately to incoming trading messages. Since the market realises only one of the many paths it could have taken, benchmarks such as LOB-Bench [5] evaluate these models distributionally over microstructure statistics.

However, distributional realism measures what a model generates, not what it has learned. A model that reproduces the statistics of order flow may have formed an internal representation of the market, or only learned regularities of the token stream that the benchmark rewards, and the score cannot tell the two apart.

Which of the two it is determines what further work should target and how far the model can be trusted in a trading system. Likelihood training has built such representations before: Transformers trained only on move sequences hold a linearly decodable representation of the Othello and chess board [6–8]. Whether a model of real order flow builds one of the market is open. In the State-Space Models (SSMs) that now lead order-flow generation [5, 9], the place to look is the recurrent state, the single fixed-size vector through which the past reaches the next prediction.

Furthermore, while next-message likelihood is theoretically exact, since by the chain rule a perfect next-message model reproduces the distribution of whole sequences, this approach remains imperfect: next-token prediction creates compounding errors over long horizons, an effect known as exposure bias [10, 11], slowly drifting to unrealistic scenarios. The cause lies in how the objective is computed. Cross-entropy scores each message against the one the market sent, conditioned on the real history, so the model is never trained on its own output and never scored on a generated sequence as a whole. The objective also leaves no way to ask the model to focus on a particular market feature or behaviour. Introducing a second training stage that scores whole rollouts can in theory optimise any such property.

Here, adversarial methods offer a solution: train a critic to distinguish real LOB sequences from generated rollouts, and update the pre-trained model to fool it. This method implicitly estimates a divergence between the two sequence distributions and, unlike hand-crafted statistics, continually reallocates capacity to whichever discrepancies the generator currently exhibits. The obstacle is that rollouts are executed through the exchange's matching engine, which is non-differentiable: no gradient connects the critic's score back to the generator's parameters. We therefore turn to optimisers that require only scalar rewards evaluated on rollouts, never a backward pass through the simulator. Evolution Guided GeneRal Optimisation via Low-rank Learning (EGGROLL) [12] perturbs the weights and differentiates nothing, and Group Relative Policy Optimization (GRPO) [13] differentiates only the generator's own log-probabilities. Current adversarial simulators avoid this

obstacle rather than solving it, either learning a differentiable stand-in for the exchange or training on flow that never reaches a book [14–16]. The value of such a stage is then decided not only by how much the targeted property improves, but also by whether it slows the compounding of errors and whether the model keeps intact what it had learned about the market.

This thesis therefore asks two questions of foundation models trained on the limit order book. The first concerns what such a model represents: which market quantities its recurrent state holds after pre-training by next-message likelihood, whether the model uses them, and whether they can be read out. The second concerns what it generates: whether a second training stage, with an objective defined on whole rollouts and optimised through the matching engine, improves the generated market, and what that stage changes beyond its aggregate score.

1.1 Contributions

1. **The first mechanistic interpretability study of a limit-order-book foundation model.** We characterise what such a model represents in a real-world application: a representation of the market's flow, held in the recurrent state of an SSM, which generalises across stocks. The representation is causally steerable: steering the recurrent state shifts the model's beliefs monotonically in dose against matched-norm nulls, while its forward information about prices adds little to the visible book.

2. **An adversarial post-training framework for HFT foundation models, with improved market realism at negligible cost.** A critic model, trained online on free-running rollouts executed through the non-differentiable matching engine, steers an evolutionary search optimiser that updates the pre-trained generator. Post-training improves WS-21 by 17.6% and Mean-L_1 by 17.0% while consuming under 50 M training tokens and 12 hours of Graphics

Processing Unit (GPU) time, a significant refinement after 170 GPU-hours and ≈48 B tokens of pre-training.

3. **A study of the role of adversarial post-training.** We examine how post-training affects our models. It tackles the failure mode that motivated this thesis, slowing the compounding of errors over free-running rollouts, while leaving the pre-trained representation and the model's directional accuracy intact. Ablation experiments then allow us to define robust guidelines for the design of adversarial post-training pipelines, from what the critic observes to how reward hacking can be mitigated.

1.2 Scope

Three boundaries are set by design. Every evaluation in this thesis is observational: models are scored on the flow they choose to emit, and no order is injected into the generated market, so market impact and the profit of any trading strategy are outside its measurements. Post-training optimises one property, realism, although the framework needs only a scalar score per rollout and can be adapted to any desired metric. The two studies use two pre-trained generators of the same family rather than one, a message-only model small enough to be read at every layer and a book-conditioned model post-trained on a single instrument, since a specialist is the strongest generator available for the stock it is tested on.

1.3 Thesis Outline

The remainder of this thesis is organised as follows. Chapter 2 introduces the key concepts and definitions on which the thesis rests: the limit order book, state-space models, the evaluation of generated markets, mechanistic interpretability, and the adversarial and gradient-free optimisation methods behind post-training. Chapter 3 reviews existing literature on feature engineering from the book, generative market simulators, the interpretability of sequence models, and zero-order optimisers. Chapter 4 specifies the corpus, the tokenisation, the two pre-trained generators, the

inference loop through the matching engine, and the evaluation protocol shared by every experiment. Chapter 5 takes up the first question of this introduction, asking what a foundation model trained on the limit order book represents; Chapter 6 takes up the second, asking whether a pre-trained generator can be improved by an objective no gradient reaches and what that improvement changes beyond the aggregate score. Chapter 7 consolidates the current understanding of these models across both studies and identifies the remaining gaps to integration into high-frequency trading. Chapter 8 concludes and outlines future research directions.

2

Background

Contents

2.1 Market Microstructure 7
2.1.1 Limit Order Book (LOB) 7
2.1.2 Market Statistics 7
2.2 Pre-Training on Limit Order Books 8
2.2.1 State-Space Models 9
2.2.2 Scaling Laws 11
2.3 Evaluating Limit Order Book Realism 12
2.3.1 LOB-Bench 13
2.3.2 Observational Performance Analysis 14
2.3.3 Market Impact Analysis 15
2.4 Mechanistic Interpretability 16
2.4.1 Representational Decoding 17
2.4.2 Causal Steering 18
2.5 Generative Adversarial Networks 19
2.5.1 Wasserstein Extension 20
2.6 Post-Training Optimisations 21
2.6.1 Policy Gradient Methods 22
2.6.2 Evolution Guided GeneRal Optimisation via Low-rank Learning 23

2.1 Market Microstructure

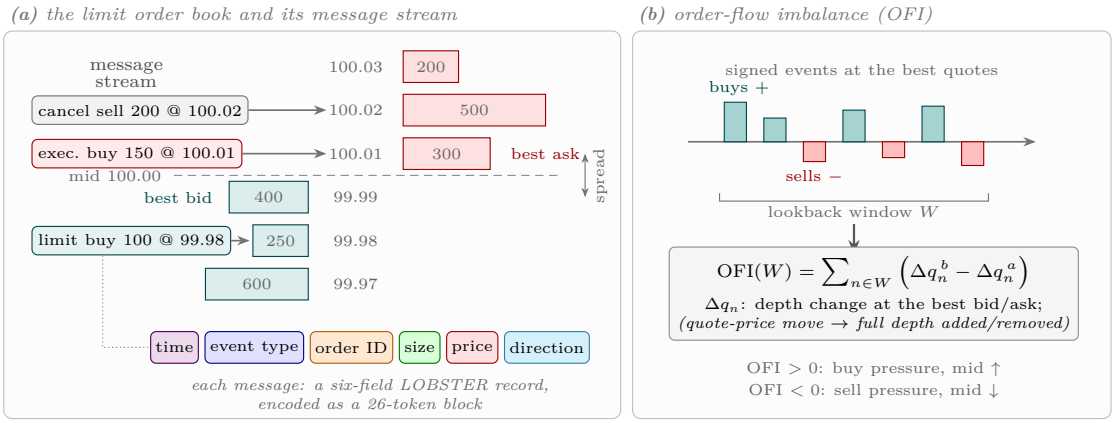


Figure 2.1: The limit order book and its message stream from records in LOBSTER format. Bar length represents volume.

2.1.1 Limit Order Book (LOB)

A LOB collects the outstanding buy (bid) and sell (ask) limit orders of a stock at discrete price levels [17, 18], with the best bid and ask (the *touch*) separated by the bid–ask spread and orders queuing under price–time priority. All activity arrives as a stream of messages at nanosecond resolution: submissions, cancellations, and executions. Each message is consumed by our model as a 26-token block.

2.1.2 Market Statistics

From the stream of messages on the LOB, one can compute the market statistics we probe for, most importantly Order-Flow Imbalance (OFI), a canonical short-horizon predictor in the microstructure literature [19, 20] largely recoverable from the message stream. OFI measures the net buy-side pressure over a lookback window of W messages:

$$e_n^b = \mathbb{1}\{P_n^b \geq P_{n-1}^b\}\, q_n^b - \mathbb{1}\{P_n^b \leq P_{n-1}^b\}\, q_{n-1}^b,$$
$$e_n^a = \mathbb{1}\{P_n^a \leq P_{n-1}^a\}\, q_n^a - \mathbb{1}\{P_n^a \geq P_{n-1}^a\}\, q_{n-1}^a,$$
$$\mathrm{OFI}(W) = \sum_{n \in W} \left(e_n^b - e_n^a\right). \quad (2.1)$$

where P_n and q_n are the best prices and total order sizes after event n. We follow [19] and compute OFI from the reconstructed book's best-quote prices and aggregate volumes after each message. Positive values correspond to net buying pressure, while negative values indicate net selling pressure. For depth k, let $e_n^{b,l}$ and $e_n^{a,l}$ denote the same expressions evaluated at the l-th best bid and ask, respectively; then $\mathrm{OFI}_k(W) = \sum_{n \in W} \sum_{l=1}^{k} \left(e_n^{b,l} - e_n^{a,l} \right)$, with $\mathrm{OFI}(W) = \mathrm{OFI}_1(W)$ the standard case above.

2.2 Pre-Training on Limit Order Books

Because the message stream is a lossless record of market activity, market simulation reduces to a sequence-modelling problem. Data feeds such as LOBSTER [4] distribute this stream at the granularity of individual messages: an autoregressive model over tokenised messages, pre-trained with cross-entropy on historical flow and combined with a matching engine, defines a fully interactive synthetic market [1, 3] (Figure 2.1). GPU-native engines make such closed-loop rollouts fast enough to train and evaluate against [21].

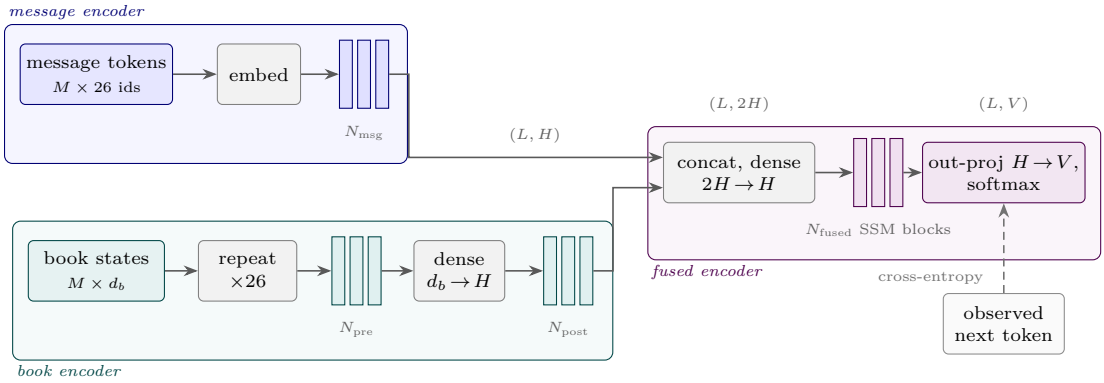


Figure 2.2: Dual-stream architecture for pre-training on order flow. Each message M is made of 26 tokens, and the book state is shared across all tokens of the same message. Here, $M = 500$, $d_b = 503$ and $V = 2112$. Message-only model variants (Chapter 5) drop the book encoder stream.

2.2.1 State-Space Models

Generating a market means running a model forward over long stretches of messages, which is where attention becomes expensive: a Transformer re-reads its entire history at every step, so both the cache it holds and the work it does grow with the length of the rollout. An SSM replaces the history with a single fixed-size vector, updated and read once per token. The memory and the work spent per token are thus constant (Figure 2.3). The gain is not confined to cost. At matched scale, state-space models match or exceed Transformer quality on downstream benchmarks [22], and the strongest generative models of order flow are built on them rather than on attention [1, 5]. This likely reflects the domain itself, whose mechanics are recurrent, the matching engine folding each message into the book one at a time. In a matched closed-loop comparison of order-flow generators, every recurrent backbone tested scores better on LOB-Bench than a Transformer of the same size, Mamba-3 by 28% in WS-21 while generating 1.36 times faster [9].

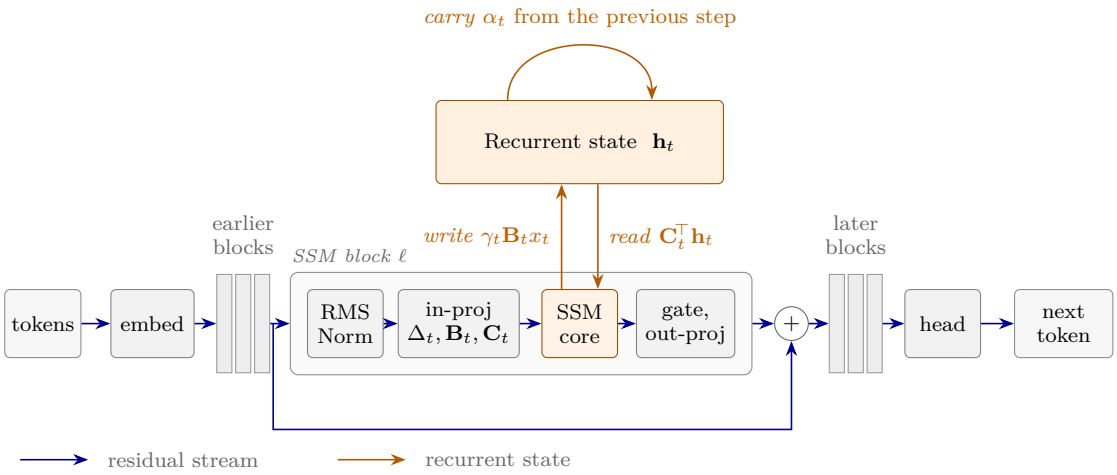


Figure 2.3: A state-space model with a focus on a single block inside the stack. The block normalises the stream, projects the token into Δ_t, $\mathbf{B}_t$ and $\mathbf{C}_t$, passes it through the recurrence, gates it and adds the result back. The recurrent state is carried from the past, attenuated by α_t, written into through $\mathbf{B}_t$ and read from through $\mathbf{C}_t$. Its size is fixed, so the cost per message does not grow with the length of the rollout.

State-space models take their formulation from linear systems theory, in which a continuous system maps an input signal to an output through a hidden state

governed by a first-order differential equation. Discretising that system with a step size Δ_t yields a recurrence over tokens,

$$\begin{aligned} \dot{\mathbf{h}}(t) &= \mathbf{A}(t)\mathbf{h}(t) + \mathbf{B}(t)x(t), & y(t) &= \mathbf{C}(t)^\top \mathbf{h}(t), \\ \mathbf{h}_t &= \alpha_t \mathbf{h}_{t-1} + \gamma_t \mathbf{B}_t x_t, & y_t &= \mathbf{C}_t^\top \mathbf{h}_t, \end{aligned} \tag{2.2}$$

with $\alpha_t = \exp(\Delta_t \mathbf{A}_t)$ and $\gamma_t = \Delta_t$, and with $\mathbf{A}_t$ a scalar multiple of the identity so that the recurrence can be evaluated by matrix multiplication [23]. Three operations are visible in the update: the state is carried forward and attenuated by α_t, the current token is written into it through $\mathbf{B}_t$, and the output is read out of $\mathbf{C}_t$.

What makes this more than a fixed linear filter is that Δ_t, $\mathbf{B}_t$ and $\mathbf{C}_t$ are projected from the current token rather than learned once. The model therefore decides, token by token, how much of the state to retain and how much of the input to admit: α_t sets the memory horizon, and a large Δ_t forgets faster while weighting the current token more heavily [22]. That data dependence is also what separates the family from linear attention, with which it shares an algebraic form. Writing the whole sequence at once gives the State-Space Duality (SSD) form $\mathbf{Y} = (\mathbf{L} \odot \mathbf{C}\mathbf{B}^\top)\mathbf{X}$, where $\mathbf{L}$ is a structured causal mask; taking $\mathbf{L}$ to be the plain triangular mask recovers causal linear attention exactly [23].

The recurrence is embedded in a block that prepares its inputs and returns its output to the network, as shown in Figure 2.3: the residual stream is normalised, projected to produce Δ_t, $\mathbf{B}_t$ and $\mathbf{C}_t$ alongside the input, passed through the recurrence, gated, projected back and added to the stream. Blocks stack, each carrying a state of its own, and the model is trained autoregressively with cross-entropy on the next token.

Mamba-3 Architecture

Mamba-3 [22] is the state-space architecture used throughout this thesis. It changes the recurrence in three ways. The discretisation is second-order accurate: the input

is carried into the state through a data-dependent convex combination of both endpoints of the step, a generalised trapezoidal rule, so that each update reads two consecutive inputs. This implicit short convolution is what allows the architecture to dispense with an explicit causal convolution over the token stream.

The remaining two changes concern the state itself. It is complex-valued: giving α_t a phase turns the update into a data-dependent rotation, computed as a rotary embedding is, which lets the model track a quantity rather than only accumulate it. Its update may also be a matrix multiplication rather than an outer product, the move from a single-input single-output system to a multi-input multi-output one; since decoding is memory-bound, the additional arithmetic performed per update buys accuracy at unchanged state size and unchanged latency.

2.2.2 Scaling Laws

In many application domains, performance of neural networks over various tasks has been shown to scale as a power law over model size, dataset size, and the amount of compute used for training [24]. The loss is commonly modelled as a function of model and data size, from which the allocation of a fixed compute budget follows [25]:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \qquad \begin{cases} N = \text{total model parameters} \\ D = \text{training tokens, repeats included} \\ E = \text{loss floor of the fitted law} \\ A = \text{scaling coefficient for model size} \\ B = \text{scaling coefficient for data size} \\ \alpha = \text{decay rate of the model-size loss} \\ \beta = \text{decay rate of the data-size loss} \end{cases}$$

In generative models trained autoregressively on the LOB, optimality has been found to differ from the allocation for Large Language Models (LLMs) [25]. While parameter count and training tokens grow at the same rate in compute-optimal

LLMs, the noisy nature of financial markets makes foundation models for high-frequency trading particularly data-hungry, with $\alpha \approx 0.76$ and $\beta \approx 0.54$ [26], although these exponents remain sensitive to the range of model sizes fitted, requiring ever more tokens given a small increase in parameter count. These results depart from the approximately linear scaling of large language models, for which $\alpha = 0.34$ and $\beta = 0.28$ [25]. Post-training on the LOB in such a noisy environment thus requires large sample sizes and carefully considered reward computation, to avoid training in directions driven by noise rather than signal.

2.3 Evaluating Limit Order Book Realism

A generative model of the book has no single correct output to score against. Divergence from the realised path is not by itself an error, since an equally plausible market could have produced a different one. What the model learns is a distribution over message streams, and it is at that level, not that of the individual path, that fidelity becomes a well-posed question. Realism is therefore measured statistic by statistic, computing a fixed set of microstructure quantities on real and on generated data and scoring the discrepancy between the two distributions for each.

It stops being sufficient the moment such a model is used to generate profits through high-frequency trading. A strategy is evaluated inside the simulator by placing its own orders, and its profit depends on what those orders do to the price. Executing size consumes resting liquidity and moves the mid-price against the trader, and this market impact is the dominant cost of trading at scale, so a simulator that gets it wrong misprices every strategy tested in it. The historical record cannot settle the question, since it holds no response to an order that was never sent, and only the simulator can supply one. Nor does the distributional comparison test it, because that comparison scores only flow the model itself chose to emit, which is correlated with the state that produced it in a way a forced order is not.

Measuring that response requires placing orders into the generated market, which the evaluations of this thesis do not do. Realism is assessed distributionally throughout, with LOB-Bench [5] as the instrument.

2.3.1 LOB-Bench

LOB-Bench [5] makes the comparison between real and generated order flow operational. A stretch of market activity is too high-dimensional to compare directly, so the benchmark reduces it first: scoring functions $\Phi_i : (\mathcal{M} \times \mathcal{B}) \to \mathbb{R}$ map a sequence of messages and the book states they produce to a single number, and the comparison is carried out one scalar at a time. Each evaluation window supplies a real prefix which the model continues, the market's own continuation is kept alongside, both pass through the same Φ, and the distributions of Φ over many windows are compared. The default scoring functions are 21 microstructure quantities, counting the two book sides and the order-flow conditionals separately. They cover the state of the book, the timing of events, where in the book new orders and cancellations fall, and traded flow; Table B.1 in Appendix B defines each.

A score can also be evaluated conditionally on a second statistic binned into deciles, asking for instance whether the model reproduces the spread it ought to show given the hour of the day. This thesis uses the unconditional scores. Conditioning splits the evaluation sample across bins, too thin to score reliably at the rollout counts our paired comparisons afford, and the unconditional aggregate is the figure reported by the closed-loop comparisons this work measures itself against [5, 9].

Two distances between the real and generated score distributions are reported:

$$L_1(p, \hat{p}) = \frac{1}{2} \sum_{b \in \text{bins}} \left| p_b - \hat{p}_b \right|, \qquad W_1(p, \hat{p}) = \int_{\mathbb{R}} \left| F(x) - \hat{F}(x) \right| \mathrm{d}x, \tag{2.3}$$

where p_b is the share of a sample falling in bin b, and F and $\hat{F}$ are the two cumulative distribution functions. L_1 is bounded in $[0, 1]$ but blind to how far misplaced mass

sits from where it belongs; W_1 measures exactly that, the average travel required to carry one distribution onto the other, which makes it responsive to the tails. It is computed on mean-variance normalised scores, otherwise incomparable across features carrying different units. The benchmark aggregates over all scores by their mean, median and inter-quartile mean, with bootstrapped confidence intervals. A discriminator trained to separate real from generated sequences sits alongside the hand-designed statistics as a worst-case scoring function, and remains able to tell the two apart after those statistics have been matched.

2.3.2 Observational Performance Analysis

A single aggregate ranks models without explaining them. Averaged over the 21 unconditional scores, it compares each statistic's distribution in isolation and over a whole window at once, so the same number can arise from different failures.

The first of these is drift within the window. An error spread evenly through a rollout and an error concentrated at its end earn the same window-level score, though only the second compounds. Reading each feature as a function of its position in the generated sequence separates them, and features degrade in characteristically different ways: some accumulate steadily with the number of messages generated, some saturate once the rollout has run long enough for the book to forget the prefix it was conditioned on, and some stay flat throughout. Where fidelity is lost is as informative as how much of it is lost.

The second is coverage of the extremes. Tail error enters an aggregate weighted by how rarely it occurs, and averaging across 21 features dilutes it further, so the tails are read directly, by comparing high quantiles of a generated distribution against the same quantiles of the real one. The book itself raises the same question of reach: whether a generated book populates as many price levels as the real one, and whether the volumes resting on them extend as far.

Section 6.6.1 reads a generated rollout the first way, position by position, and Section 7.2 reads it the second way, at the extremes.

2.3.3 Market Impact Analysis

The distributional scores of LOB-Bench (Section 2.3.1), its price response functions, and the drift and tail readings that refine them are all computed on messages the model chose to send [5]. Impact is a counterfactual, what the book does when an order is placed into it from outside. Writing B_{t+1} for the book after the next message and $m_{\le t}$ for the history before it, the two conditionings differ:

$$p\Big(B_{t+1} \mid m_{\le t},\, \mathrm{do}(m_{t+1} = a)\Big) \;\neq\; p\Big(B_{t+1} \mid m_{\le t},\, m_{t+1} = a\Big). \tag{2.4}$$

An order the model emitted is informative about the state that produced it, whereas a forced order carries no such information. The right-hand side is all a distributional benchmark can reach; the left-hand side is what a trading agent would meet.

What an intervention ought to produce is fixed by the empirical impact literature [18, 27]. A large position is sliced into child orders worked over minutes or hours, the aggregate being called a metaorder, and the price change it causes grows as a concave power of its size relative to the day's activity, consistently across equities, futures and foreign exchange,

$$I(Q) = Y\,\sigma\left(\frac{Q}{V}\right)^{\beta}, \qquad \beta \approx \tfrac{1}{2}, \tag{2.5}$$

where $I(Q)$ is the price change caused by executing a size Q, V the volume traded that day, σ the daily volatility and Y a dimensionless constant of order one [18, 28, 29]. The exponent is remarkably stable, a survey covering an entire exchange recovering the same square root for essentially every stock on it [30].

The law's concavity is a consequence of the market refilling the liquidity a metaorder consumes rather than absorbing it permanently [31]. Each child order

eats into the book and the book rebuilds behind it, so the children that follow meet a partly recovered market and move the price less than the first ones did. The rebuilding does not stop when the order does, which is why impact falls back from its peak instead of holding there, settling at around two thirds of it [32, 33]. A correct profile therefore has three parts, a concave build-up over the execution, a maximum at completion, and a partial relaxation afterwards.

Measuring that profile in a generative model realises the left-hand side of (2.4) directly: the same prefix is rolled out twice, once untouched and once with the children of a metaorder forced into the message stream, and impact at horizon H is the gap between the two mid-price paths,

$$M(H) = \Delta\mathrm{mid}_{\mathrm{with}}(H) - \Delta\mathrm{mid}_{\mathrm{without}}(H). \tag{2.6}$$

Pairing the rollouts makes M causal, since the generator's own drift enters both arms alike. Part of the gap is mechanical, the forced order consuming resting volume as it fills; only the rest, the model's reaction to having seen it, reflects what the model learned [34]. The curve is then read against four requirements: the sign of the trade, growth with its size, concavity with an exponent near one half, and relaxation once execution ends.

2.4 Mechanistic Interpretability

The LOB-Bench scores of Section 2.3.1, the drift and tail readings that refine them, and the impact curve of Section 2.3.3 all summarise what the model emitted, and none of them says how it was produced. The distinction drawn in Chapter 1 between a model that has formed a representation of the market's state and one that has only fitted the token stream is invisible to them. This difference matters where the simulator is put to work: asked to continue from a regime the training data covered thinly, or to react to an order placed into it from outside, a model falls back on whatever it has built internally. Mechanistic interpretability turns the

question inward, asking which quantities a model computes and holds as it reads its input [6–8, 35], and whether those quantities drive what it predicts next [36, 37].

A state-space model is a direct subject for that question. Its recurrence carries one vector of fixed size from token to token (Section 2.2.1), and that vector is the only route by which the distant past reaches the next prediction. The state is a compression of the history into a fixed budget, so what a model keeps under that budget is a statement about what it treats as the state of the market. The architecture also separates two internal quantities that answer different questions. The residual stream carries the token currently being processed and reflects the present message, while the recurrent state accumulates across the window and holds what survives of the past. Locating a market quantity in one rather than the other distinguishes what the model reads off the last message from what it has built up over the window.

2.4.1 Representational Decoding

Representational decoding tests whether a market quantity can be read out of a model's activations. Let $\mathbf{z} \in \mathbb{R}^d$ be the activation collected at a chosen point in the network, the residual stream or the recurrent state, and let y be a statistic of the market the model has just read, such as order-flow imbalance or the spread. The readout is a linear map $\mathbf{w}$, the form the model's own layers use to read the state through $\mathbf{C}_t$ in (2.2). Over n windows it is fitted by ridge regression,

$$\hat{\mathbf{w}} \;=\; \arg\min_{\mathbf{w}} \sum_{i=1}^{n} \big(y_i - \mathbf{w}^\top \mathbf{z}_i\big)^2 \;+\; \lambda \|\mathbf{w}\|_2^2. \tag{2.7}$$

The penalty $\lambda\|\mathbf{w}\|_2^2$ keeps the weights small. It is needed because the state has many more dimensions than there are windows to fit on, and an unpenalised map could reproduce any target exactly, describing the fit rather than the model.

Quality of fit is measured on windows the map was not fitted on, by the coefficient of determination R^2, computed as

$$R^2 \;=\; 1 \;-\; \frac{\sum_i \big(y_i - \hat{\mathbf{w}}^\top \mathbf{z}_i\big)^2}{\sum_i \big(y_i - \bar{y}\big)^2}, \qquad \bar{y} = \frac{1}{n}\sum_i y_i. \tag{2.8}$$

This value represents the share of the target's variance the readout explains: one is an exact readout, zero a state that adds nothing over the target's mean. Because market statistics are measured in shares, ticks and seconds, this normalisation is what makes scores comparable across features, layers and stocks.

None of these quantities is supervised. The model is trained to predict the next token of the message stream and nothing else, so a statistic that can be read from its state is an intermediate result the objective made worth holding. Which statistics appear is therefore informative in itself.

2.4.2 Causal Steering

Decoding finds what the state contains, not what the model does with it. A quantity can be recoverable and still be inert, computed on the way to something else and never consulted when the next message is chosen [36]. Telling the two apart means acting on the model rather than observing it: change the quantity inside the state by hand, then see whether the predictions move the way its meaning says they should.

A feature is carried by a direction. Writing $\mathbf{v}$ for the direction in state space along which the feature varies, steering displaces the state at the point of intervention,

$$\mathbf{h} \;\longleftarrow\; \mathbf{h} + \alpha\,\mathbf{v}, \tag{2.9}$$

and lets the model continue from there [37, 38]. The scalar α is the dose: its sign selects which way along $\mathbf{v}$ the state is moved and its magnitude how far, in units of the feature's spread across real windows.

Three properties of the response are informative. Its *sign and magnitude* test whether the model's encoding agrees with the market's, a displacement towards

buy-side pressure having to raise the probability that the next message is a buy, and to raise it further as α grows. Its *specificity* tests whether the effect follows the encoded direction rather than the displacement itself, since a large enough push in any direction disturbs the output. Its *reach* tests how far the effect propagates, from the model's expectation of the next message through to the market it generates: a shift that appears in the first and not the second locates the break between what the model predicts and what it emits.

2.5 Generative Adversarial Networks

A Generative Adversarial Network (GAN) trains a generator against a discriminator that learns to separate real from generated samples [39]. The discriminator is fitted to tell the two apart, the generator to defeat it, and the pair is optimised as a single minimax problem,

$$\min_{G}\ \max_{D}\ \mathbb{E}_{x\sim p}\Big[\log D(x)\Big]\ +\ \mathbb{E}_{\hat{x}\sim \hat{p}}\Big[\log\Big(1-D(\hat{x})\Big)\Big], \tag{2.10}$$

where p is the distribution of real samples and $\hat{p}$ the one the generator induces.

The reason to arrange training this way is what the inner maximisation produces. For a fixed generator, the discriminator that maximises (2.10) is $D^{\star}(x) = p(x)/\{p(x)+\hat{p}(x)\}$, and substituting it back reduces the generator's problem, up to a constant, to minimising the Jensen–Shannon divergence between p and $\hat{p}$ [39]. The discriminator is therefore a learned divergence: nothing has to be said in advance about which features of a sample matter, and because the divergence is re-estimated as training proceeds, it tracks whichever discrepancy the generator currently exhibits rather than the one an author anticipated.

This benefit comes at the cost of stability. Two networks are optimised at once, so no single objective is descending and the pair can circle each other instead of settling. A generator can also lower its loss by giving up on the parts of the

distribution it finds hardest, reproducing a few modes convincingly rather than all of them. The divergence adds a third difficulty. When p and $\hat{p}$ barely overlap, the discriminator separates them almost perfectly, the Jensen–Shannon divergence sits at its maximum nearly everywhere, and its gradient with respect to the generator vanishes, so the better the discriminator becomes the less the generator learns [40].

2.5.1 Wasserstein Extension

The Wasserstein formulation keeps the architecture and changes the divergence. The Wasserstein-1 distance is the least average distance over which mass must be carried to turn one distribution into the other, and Kantorovich–Rubinstein duality rewrites that transport problem as a search over 1-Lipschitz functions,

$$W_1(p,\hat{p}) = \inf_{\gamma\in\Pi(p,\hat{p})} \mathbb{E}_{(x,\hat{x})\sim\gamma}\Big[\|x-\hat{x}\|\Big] = \sup_{\|f\|_L\le 1} \mathbb{E}_{x\sim p}\Big[f(x)\Big] - \mathbb{E}_{\hat{x}\sim\hat{p}}\Big[f(\hat{x})\Big], \quad (2.11)$$

where $\Pi(p,\hat{p})$ collects the joint distributions having p and $\hat{p}$ as marginals. This is the same distance as the W_1 of (2.3), which the benchmark computes directly because each of its scores is one-dimensional. Over whole sequences it cannot be computed, only estimated, and the right-hand side says how: find the 1-Lipschitz function whose mean output differs most between real and generated samples.

The network that performs that search is a critic rather than a classifier [40]. It returns a real number instead of a probability, and the gap between its averages on real and on generated samples estimates the distance. This is what removes the vanishing gradient: a critic reports how far apart the two distributions are rather than whether it can tell them apart, so it still says something useful when they barely overlap. The duality holds only over 1-Lipschitz functions, so the critic must be held to that class, either by penalising its gradient norm [41] or by dividing each layer's weights by their largest singular value [42].

Two devices then keep the networks from oscillating. Giving the critic and the generator different learning rates is enough to converge to a local equilibrium under

mild conditions [43]. Moreover, the zero-centred penalty $R_1 = \frac{\gamma}{2}\,\mathbb{E}_{x\sim p}\Big[\|\nabla_x D(x)\|^2\Big]$ flattens the critic on real samples. Its purpose is to make the solution stable: once the generator has matched the data, the critic is slowed to avoid pushing the generator away and oscillating around the answer instead of stopping at it [44].

The critic yields an objective, though not a means of training on it. Generated messages are discrete and replayed through a matching engine, so no gradient runs from the score back to the weights. Optimisation must therefore proceed from the score alone; Section 2.6 turns to the methods that allow it.

2.6 Post-Training Optimisations

Cross-entropy scores the next message against the one the market really sent, whereas realism is a property of a whole rollout, established after the fact by a critic or a benchmark. Post-training optimises full sequences directly, and the difficulty is that it can only be obtained by running the model: the rollout passes through sampling and through the matching engine, neither of which admits a derivative, so no gradient links the score back to the weights that produced it.

Two families of estimator build an update out of such evaluations, differing in what they randomise. Policy-gradient methods randomise the model's output. The model samples, each sample is scored, and the model's own log-probability of what it emitted is differentiated so that probability mass moves towards the samples that scored well. Zero-order methods randomise the parameters instead. Perturbed copies of the model are each scored, and the perturbations are combined in proportion to their scores [45, 46]. Neither family differentiates the environment. What separates them is what is left to differentiate: a policy gradient still requires a backward pass through the network, while a zero-order update requires none and treats the model as a black box.

2.6.1 Policy Gradient Methods

Policy-gradient methods score each sampled sequence and raise or lower its log-likelihood accordingly, shifting probability mass towards higher-reward samples. Writing π_θ for the model and $R(x)$ for the score of a completed rollout x, this rests on the identity

$$\nabla_\theta \, \mathbb{E}_{x\sim\pi_\theta}\Big[R(x)\Big] \;=\; \mathbb{E}_{x\sim\pi_\theta}\Big[R(x)\,\nabla_\theta \log \pi_\theta(x)\Big], \tag{2.12}$$

which turns an expectation the model cannot differentiate into one it can [47]. Because $\log \pi_\theta(x)$ is a sum over the tokens of the rollout, the update reaches each token separately. The estimator is an average over sampled rollouts and is noisy, and subtracting a baseline from R leaves it unbiased while reducing that noise; what remains after the subtraction is the advantage.

Group Relative Policy Optimization

Reusing a batch of rollouts for several updates would amortise their cost, but (2.12) holds only for samples drawn from the current π_θ. Proximal Policy Optimization (PPO) [48] resolves the tension by weighting each token by the ratio between its probability now and under the policy that produced the batch, clipped so that no token is pushed far from where its advantage A_t was measured, a trust region imposed by the objective rather than by a constraint on the optimisation [49]. GRPO [13] keeps that machinery and removes the value network. The baseline in A_t is ordinarily a second model trained to predict the score from the current state, fitted on the same noisy signal as the policy and awkward to fit at all when the score exists only once a rollout is complete. GRPO takes the baseline from the group instead: several rollouts are drawn from the same context and each is given the advantage $A_i = (R_i - \mathrm{mean}_j\, R_j)/\,\mathrm{std}_j\, R_j$, shared by every token it contains. The group mean is the baseline and the group's spread sets the scale, so a context on which every rollout scores alike contributes nothing to the update, and differences in difficulty between contexts cancel instead of having to be learned.

Two things must hold for any of this to reach a market generator. The score must attach to what the model emitted, which a discriminator supplies by judging rollouts as they are produced [50]. The model must also stay near the generator it started from, which nothing in the objective so far enforces. A learned reward is a proxy, and optimising it far enough finds the places where it parts company with realism, a failure known as reward hacking. The remedy is the one PPO and GRPO already carry, a penalty on divergence from the pre-trained reference, and it is what we take into our pipeline.

2.6.2 Evolution Guided GeneRal Optimisation via Low-rank Learning

Evolution Strategies (ES) sample perturbations of the parameters, score each perturbed model on the objective, and combine the perturbations, weighted by rank-shaped antithetic scores, into an update [51]. Three properties recommend the family for post-training. It is zero-order, placing no differentiability requirement on the objective or on anything standing between the parameters and the score. It is robust to noise, since population-based exploration smooths irregularities in the landscape and tolerates the discontinuities and ill-conditioned curvature that trouble gradient estimates over long rollouts [46]. It parallelises cleanly, because fitness evaluations are independent across population members, so workers exchange one scalar each rather than aggregate full gradient tensors across devices.

What has confined ES to small models is the cost of the perturbations themselves. A naive implementation materialises a full-rank perturbation $E \in \mathbb{R}^{m\times n}$ of every weight matrix $M \in \mathbb{R}^{m\times n}$ for each population member, a copy of the whole parameter set per member evaluated through its own forward pass. Memory grows as mn per layer per member and compute with population size times sequence length, so both costs dominate long before the billion-parameter regime [12].

EGGROLL [12] makes this efficient by drawing each perturbation as a rank-r outer product (collapsing per-member memory from $O(mn)$ to $O(r(m+n))$) and

competitive with other policy optimisation methods used to fine-tune modern LLMs. Factors A and B are sampled entry-wise and the search matrix is their scaled outer product:

$$E = \frac{1}{\sqrt{r}} AB^\top, \qquad A \in \mathbb{R}^{m\times r}, \quad B \in \mathbb{R}^{n\times r}, \quad r \ll \min(m, n). \tag{2.13}$$

Each entry of $AB^\top$ is a sum of r products, so the $1/\sqrt{r}$ factor is what holds the variance of E at $O(1)$ as the rank grows. The entries of A and B are drawn independently from a zero-mean, symmetric, absolutely continuous distribution p_0 with finite fourth-order moments and variance σ_0^2, and need not be Gaussian. The perturbation carried by any one member is low-rank, but the update is not: averaging N rank-r matrices yields an update of rank $\min(Nr, m, n)$, which is full rank at any realistic population size.

The perturbations are combined in proportion to their fitness. With a population of N workers, step size α_t, perturbation scale σ and fitness f, the update applied is

$$M_{t+1} \leftarrow M_t + \frac{\alpha_t}{N} \sum_{i=1}^{N} E_{i,t}\, f(W = M_t + \sigma E_{i,t})\,, \tag{2.14}$$

which Appendix A derives from the score of the low-rank perturbation distribution: as the rank grows that distribution tends to a matrix Gaussian whose score is $-E$, and (2.14) is the Gaussian ES update with that score substituted. Low rank is not a concession on update quality. The low-rank estimator departs from the full-rank Gaussian ES gradient by $O(1/r)$, faster than the $O(1/\sqrt{r})$ the central limit theorem alone would give, so the approximation is already accurate at ranks in the single digits [12].

The factorisation also changes what the forward pass asks of the hardware. For a batch of activations v_i, naive ES computes $v_i(M + \sigma E_i)^\top$ as a batched matrix multiplication in which every entry of $M + \sigma E_i$ is used exactly once, so arithmetic

intensity is poor and GPUs run far below peak. Under (2.13) the same quantity factors as

$$v_i \left(M + \sigma E_i\right)^\top = v_i M^\top + \frac{\sigma}{\sqrt{r}} \left(v_i B_i\right) A_i^\top, \tag{2.15}$$

where the bulk of the work is one ordinary matrix multiplication against the shared M, followed by a cheap low-rank correction per member. This is the decomposition behind efficient batched Low-Rank Adaptation (LoRA) [52] serving, and it is why a whole population evaluates at close to the throughput of plain batched inference. Beyond lowering communication costs, low-rank factorisation also cuts memory: workers are initialised from known seeds deterministically, so each reconstructs any factors on demand without storing perturbation in memory. Algorithm 1 states the procedure.

Algorithm 1 EGGROLL [12]

Require: rank r, step size α, perturbation scale σ, steps $T_{\max}$, population size N

Initialise M and the N workers with known random seeds ς
for $t = 1, \ldots, T_{\max}$ **do**
    for each worker $i \in \{1, \ldots, N\}$ in parallel **do**
        $A_i \sim p(A_i), \quad B_i \sim p(B_i)$
        $E_i \leftarrow \frac{1}{\sqrt{r}} A_i B_i^\top$
        $f_i \leftarrow f(W = M + \sigma E_i)$
    end for
    Workers exchange the scalar fitnesses f_i
    for each worker $i \in \{1, \ldots, N\}$ in parallel **do**
        Reconstruct E_j for $j \in \{1, \ldots, N\}$ from ς
        $M \leftarrow M + \frac{\alpha}{N} \sum_{j=1}^{N} E_j f_j$
    end for
end for

3

Related Work

Contents

3.1 Feature Engineering for Market Predictions 27
3.2 Generative Models for Market Simulation 28
3.2.1 Agent-Based Methods 28
3.2.2 Learned World Agents 28
3.2.3 Autoregressive Foundation Models 29
3.3 Mechanistic Interpretability of State-Space Models . . 30
3.3.1 Representation and Causal Use 30
3.3.2 Interpretability of State-Space Models 30
3.3.3 Interpretability of Financial Models 31
3.4 Post-Training and Zero-Order Optimisation 31
3.4.1 Adversarial Training for Sequence Models 32
3.4.2 Policy Optimisation Methods 32
3.4.3 Hardware-Aware Evolutionary Search 32
3.5 Positioning 33

3.1 Feature Engineering for Market Predictions

For decades, prediction from the LOB proceeded by distilling the book into a handful of hand-designed signals. The canonical example is OFI, which aggregates the net change in queue sizes at the touch into a single quantity and explains short-horizon price changes through a near-linear and remarkably stable relationship [19], within a broader tradition of parsimonious descriptions of how order flow moves prices [27, 28] surveyed in [17, 18]. Later work widened the aperture without abandoning the design step, extending the statistic beyond the touch to deeper price levels and across the cross-section of related assets [53], and learning non-linear combinations of multi-level imbalances to extract alpha at several horizons [20].

Deep learning changed the estimator, not the input. DeepLOB, TransLOB and their successors all consume the same object as the hand-designed signals, a periodically sampled snapshot of the book truncated to its top k price levels, and differ only in what they fit to it: convolutional and recurrent layers to classify mid-price direction [54], a Transformer in place of the recurrence [55], or a single model shared across many stocks [56]. Choosing k, the sampling frequency and the quantities retained is feature engineering performed before training, and once the evaluation protocol is held fixed across models and datasets the gains it delivers prove fragile [57].

An emerging line of work removes that design step by consuming the message stream itself rather than a projection of the book: every submission, cancellation and execution becomes a token, and the book is reconstructed by the matching engine rather than supplied as an input [1, 3]. Nothing is selected and nothing is discarded. How that stream should be tokenised remains open. Field-level encodings assign a fixed block of tokens to each message [1], while byte-level schemes drop the tokeniser entirely [58].

Removing the design step raises the ceiling performance of deep models. A hand-designed feature guarantees that a known signal reaches the model but bounds what else can: the top ten price levels cannot convey what they discard. Raw message input removes that bound while keeping the engineered signals reachable, since OFI and its relatives are functions of the same stream and a model should reconstruct them to the extent that they prove useful [19, 20]. Whether it does is a question about its internals rather than its outputs, which we take up in Chapter 5.

3.2 Generative Models for Market Simulation

A simulated market must satisfy two requirements at once. It has to reproduce the statistical texture of real order flow, and it has to stay interactive: an order submitted into it should change what happens next, as it would on a real exchange. Successive generations of simulators have traded one against the other.

3.2.1 Agent-Based Methods

Simulators such as ABIDES [59] compose hand-crafted trading agents around an exchange model; they are interactive by construction but struggle to match the fine structure of real order flow. These simulators are accurate in their mechanics and inaccurate in their flow. The matching engine captures how the book responds to an order independently of who submits the orders around it, whereas the agents submitting them reproduce only the behaviours their rules anticipate. Later work kept the loop and made it fast, with GPU-native engines running many rollouts in parallel for large-scale training [21].

3.2.2 Learned World Agents

A data-driven line therefore replaces the agent pool with a learned world agent: Stock-GAN generates order streams with a conditional Wasserstein GAN, sidestepping the non-differentiable exchange by learning a neural approximation of the matching mechanism [14]; Coletta et al. develop CGAN world agents deployed inside multi-agent simulations [15, 60] and analyse their explainability and robustness [61];

Hultin et al. factorise message generation into a chain of conditional distributions [62]; and TRADES brings denoising diffusion to the same world-agent setting [16]. The pattern recurs across GAN [63] and diffusion [64, 65] formulations alike.

What unites them is the compromise each has to make with the exchange. An adversarial or denoising objective is defined on whole sequences, so training it end-to-end requires gradients to flow back through the matching engine, which is a discrete, rule-based procedure that admits none. Every model above resolves this the same way, by replacing the engine with a differentiable surrogate or by generating flow that is never fed back into a book. While the objective is accurate, the environment is nonetheless only an approximation of the true one.

3.2.3 Autoregressive Foundation Models

More recently, token-level autoregressive foundation models have become the strongest generative simulators. Trained on tokenised message streams with next-token cross-entropy, they scale with data and parameters in the manner familiar from language modelling [1–3, 66, 67], and LOB-Bench [5], which scores real against generated rollouts over a battery of microstructure statistics, finds them ahead of GAN-based and parametric alternatives. Because generation is one message at a time, the true matching engine can sit inside the loop: nothing has to be approximated to obtain a full rollout.

The two lines therefore have complementary blind spots. The adversarial approach optimises the right objective, a divergence between real and generated distributions that reallocates capacity to whichever discrepancy the generator currently exhibits, but pays for it with a surrogate exchange. The autoregressive approach keeps the real exchange and the entire message stream, but optimises next-token likelihood, an objective that never observes a rollout.

This thesis keeps the real exchange and changes the objective: we take a pre-trained foundation model as given and ask how a second, adversarial training stage

improves it, which means optimising an objective that no gradient connects to the generator's parameters (Section 3.4).

3.3 Mechanistic Interpretability of State-Space Models

While most prior literature focuses on mechanistic interpretability in transformer-based models, recent work has also shown that autoregressive models can develop internal representations of their environment throughout pre-training. A Transformer trained only on Othello moves builds an internal board representation [6], later shown to be linearly decodable [7], with similar findings in chess [8] and program execution [68]. Beyond games, sequence models encode static real-world properties such as protein structure [69] and spatial and temporal coordinates [70].

3.3.1 Representation and Causal Use

Developing an internal representation of the environment in the recurrent state says nothing about its causal use. Probing establishes only what a state contains [36], and its verdicts have repeatedly proved fragile: OthelloGPT's board representation rests on context-specific heuristics rather than one coherent board state [71], models with strong predictive accuracy hold implicit maps that prove incoherent under systematic evaluation [72, 73], and such maps break down on adversarially chosen sequences [74]. Decoding performance can thus overstate the coherence of the underlying world model. The response has been interventional: rather than read a representation, one writes into it and checks that the output moves as its meaning dictates, developed as activation addition and steering vectors for language models [37, 38]. Without it, an encoded quantity such as OFI may be a side effect of learning, causally inert during market simulation.

3.3.2 Interpretability of State-Space Models

Against this standard, mechanistic interpretability of SSMs remains vastly under-explored. Most work on interpreting them is attributional rather than mechanistic:

recasting Mamba's selective scan as an implicit attention map [75], decomposing it into token-to-token interactions [76], and propagating relevance through the recurrence [77] all establish which inputs a prediction depends on, not what the model represents internally. The few studies that do read and intervene on the recurrent state are confined to language: locating and editing factual associations in Mamba by causal tracing [78], asking whether Transformer mechanisms recur in recurrent architectures [79], transferring steering vectors and the tuned lens from Transformers [80], and fitting Sparse AutoEncoders (SAEs) to the state itself [81].

3.3.3 Interpretability of Financial Models

The models of Section 3.2 are assessed almost entirely from the outside. Understanding them beyond distributional realism and predictive accuracy is currently restricted to post-hoc attribution, which indicates *which* inputs influence a prediction but not *how* they are processed. Examples include sensitivity analyses of DeepLOB [54], attention-based reading of the book [82], and robustness analyses of generative LOB models [61]. Mechanistic analysis in finance has so far largely targeted language models applied to financial text [35, 83, 84], including concept steering [85], while interventions on time-series foundation models steer generic signal properties [86] or transplant bulk activation statistics between market regimes [87]. Closest to our study, concurrent work probes the latent volatility state of a Transformer trained on simulated stochastic-volatility returns [88], not a foundation model of real order-book messages. To our knowledge, no prior work applies mechanistic interpretability to a foundation model trained natively on limit-order-book data, nor probes or steers an SSM's recurrent state outside of language and synthetic game domains. There is thus no prior work to build on for real-world applications such as market simulation.

3.4 Post-Training and Zero-Order Optimisation

Prior work supplies the two ingredients this thesis combines: a two-stage recipe in which a pre-trained generator is refined against an adversary, and optimisers that

ask nothing of the environment beyond a scalar score per rollout.

3.4.1 Adversarial Training for Sequence Models

For discrete sequences, discriminator gradients cannot flow through sampling; SeqGAN resolved this by treating generation as a policy updated with REINFORCE against discriminator rewards [50]. The same two-stage shape recurs in adversarial post-training of diffusion models, where a pre-trained model is refined against real data after likelihood-style pre-training [89]: pre-train for coverage, then optimise an adversarial objective for fidelity. In both, the second stage optimises against a learned adversary rather than a fixed objective.

3.4.2 Policy Optimisation Methods

Post-training pre-trained models against scalar rewards is standard practice in the LLM literature, via policy-gradient methods such as GRPO [13], now part of the standard recipe for open large language models [90]. These pipelines share a shape: the reward comes from a model fitted offline and then frozen, and the update is held near a reference policy by a Kullback–Leibler (KL) penalty. The constraint is necessary because any learned reward is eventually over-optimised [91], the measured score continuing to rise while true quality falls.

3.4.3 Hardware-Aware Evolutionary Search

Evolution strategies estimate a search direction from the scores of sampled parameter perturbations, requiring no gradient of the objective [45, 46], and rival reinforcement learning on control tasks with a structure that parallelises across workers exchanging only scalar rewards [51]. Estimator variance grows with the number of parameters perturbed, however, so the population needed was expected to grow with the model, and with it the cost of evaluating one.

Recent work removes that cost. Drawing each perturbation as a low-rank outer product turns a whole population's forward passes into one batched matrix multiplication, making large populations affordable at scales where the dense

formulation was not [12], and on that footing evolution strategies fine-tune large language models competitively with reinforcement learning [92]. None of this work applies the method to a generator whose rollouts pass through a simulator, or against a reward that is itself learned and moving, or at a matched rollout budget against a policy-gradient alternative.

3.5 Positioning

The literature reviewed above leaves two gaps, one on each question of this thesis. The representational studies closest to Chapter 5 are the emergent-world-model results for Transformers trained on Othello and chess moves [6–8] and, in finance, the concurrent probe of a Transformer trained on simulated stochastic-volatility returns [88]. While they establish that likelihood training can build a representation of the environment, they restrict themselves to Transformer-based models, and no financial model tests causal use of decoded features. A foundation model trained on the limit order book has not been read, nor has the recurrent state of a state-space model outside language models. Chapter 5 does both, reading the state of an order-flow generator and testing by intervention whether what it holds is used. On post-training, the autoregressive models that LOB-Bench ranks first optimise next-token likelihood, an objective that never scores a generated sequence as a whole [1, 5], while the adversarial and diffusion world agents optimise a sequence-level objective through a surrogate of the exchange or on flow that is never fed back into a book [14–16]. The two-stage recipe of Section 3.4, pre-train for coverage and refine against an adversary, has not been applied to a generator whose environment admits no gradient. Chapter 6 does so, optimising a sequence-level objective through the true matching engine with zero-order optimisers and measuring what the stage changes beyond its score.

4

Methodology

Contents

4.1 Dataset 35
4.1.1 LOBSTER 35
4.1.2 Corpus and Splits 35
4.2 Tokenisation 37
4.3 Pre-Trained Generators 38
4.3.1 Model Architecture 38
4.3.2 Pre-Training Recipe 40
4.4 Inference Pipeline 41
4.5 Evaluation Protocol 42
4.6 Training Environment 43

4.1 Dataset

All experiments in this thesis draw on a single corpus of message-level limit order book records covering eight US large-capitalisation equities over four years. Each study operates on a restriction of that corpus, narrowed by instrument, by input stream, or by date. This section specifies the record format, the corpus built from it, and the restrictions that define what each model is trained and evaluated on.

4.1.1 LOBSTER

The LOBSTER [4] format contains message-level information about individual equities. It provides event types, 10^{-4} USD prices, volume, IDs, and nanosecond-resolution timestamps. We discard out-of-hours messages, hidden executions, cross trades, trading-halt indicators, and clip volumes at 9,999 shares.

4.1.2 Corpus and Splits

The corpus covers eight US large-capitalisation equities, GOOG, AAPL, NVDA, AMZN, META, TSLA, MSFT and AMD, referred to below as MegaCap-8, from January 2022 to December 2025. Each trading day contributes its full message stream together with the book state that stream induces. The stream is cut at a stride of 500 messages from a random per-day offset, so one window is 500 messages, or 13,000 tokens under the encoding of Section 4.2.

Two restrictions in width are taken from it. Chapter 5 withholds the book, so that any book statistic recovered by a probe has been reconstructed from order flow rather than read from the input. Chapter 6 restricts the corpus to GOOG and retains the book, which the matching engine returns after every generated message, and which concentrates the adversarial signal on one instrument's microstructure. Table 4.1 collects the restrictions and the chapter each supports.

Table 4.1: Restrictions of the corpus. The model of Chapter 5 is given the message stream alone, while that of Chapter 6 receives the book alongside it. The final column counts training tokens for the top half and held-out 500-message evaluation windows for the bottom half.

Chapter	Usage	Stocks	Time period	
Training				Tokens
Chapter 5	Pre-training	MegaCap-8	2022–2025	$\approx 63\,\mathrm{B}$
Chapter 6	Pre-training	GOOG	2022–2025	$\approx 48\,\mathrm{B}$
Chapter 6	Post-training	GOOG	2022–2025	$\approx 50\,\mathrm{M}$
Evaluation				Windows
Chapter 5	Probing panel	GOOG	2026-01	2,014
Chapter 5	Generalisation	MegaCap-8, JPM, ORCL	2026-01	2,014/stock
Chapter 6	Sealed test	GOOG	2026-01/02	4,096

Post-training data is sourced from the same period as the pre-training data but restricted to fully unseen windows, a candidate being admitted only if the entire span it covers, conditioning prefix and continuation together, intersects no window the anchor consumed. No other construction isolates the objective in this way. Post-training on windows the anchor has already fitted would leave two explanations for any improvement, the new objective and a second pass over data the model has already learned, with no way to separate them. Post-training on a later period would place the treatment data closer in time to the test panel than the anchor's data, so a gain measured on that panel could reflect the shorter gap rather than the objective.

Held-out evaluation data is drawn from the most recent period of the record, later than anything either training stage reaches, which rules out leakage by construction. No message dated after 31 December 2025 enters either training stage or takes any part in checkpoint selection. Chapter 5 evaluates on GOOG over January 2026, 2,014 windows across 20 trading days, and extends the same panel to the seven remaining corpus equities and to JPM and ORCL, two equities absent from the corpus, which separates what the model has learned about an instrument from what

it has learned about the market. Chapter 6 evaluates on GOOG over January and February 2026, 4,096 windows across 28 trading days, scored once.

The test period shows no dramatic regime shift. Event composition and volatility are stable across periods, while activity and price trend upward through the training years (Table 4.2).

Table 4.2: Statistics for the GOOG stock over the training years, the four quarters of 2025, and the January–February 2026 test period.

Period	Msgs/day (M)	Exec. (%)	Spread (bps)	Daily RV (%)	Mid ($)
2022–2025	4.41	1.45	1.18	1.44	157
2025 Q1	5.95	1.20	1.25	1.50	183
2025 Q2	6.87	1.55	1.20	1.80	165
2025 Q3	5.15	1.40	1.14	1.31	210
2025 Q4	8.03	0.88	1.48	1.56	286
Test	6.17	1.29	1.24	1.70	329

4.2 Tokenisation

LOBSTER messages are each encoded as 26 tokens over a shared vocabulary of $|\mathcal{V}| = 2{,}112$ (Figure 4.1), with four reserved special tokens (mask, hidden-event, not-applicable, start-of-sequence). Fields cover the message's event type, direction, timing, price, and size, plus the referenced order's attributes where applicable; sizes and price magnitudes are split into digit-token pairs (base-100 and base-1,000 respectively), shrinking the vocabulary while covering ±999,999 ticks. Prices are encoded in ticks relative to the mid-price before the message. Absolute timestamps are kept in the loss for temporal consistency but are filled deterministically at inference. Logit masking enforces each position's valid-token set, guaranteeing syntactic validity (e.g., event types in $\{1, 2, 3, 4\}$) though not semantic validity (e.g., cancelling an order absent from the book).

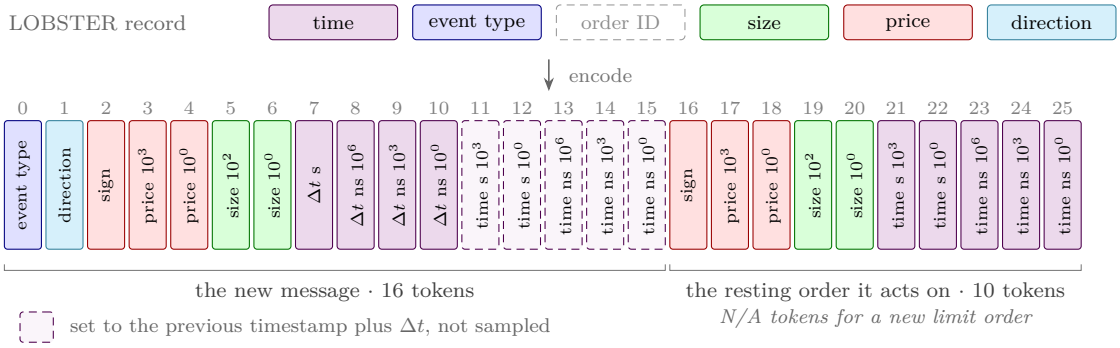


Figure 4.1: The 26-token encoding of one LOBSTER message from a shared vocabulary of 2,112 ids. Prices are signed and split into two base-1,000 digits, sizes into two base-100 digits, and both time fields into base-1,000 digits. The order ID is never tokenised. The dashed positions carry the absolute timestamp, which is kept in the training loss but reconstructed at inference from the previous timestamp and the generated Δt.

4.3 Pre-Trained Generators

Every experiment in this thesis starts from a generator pre-trained by next-token prediction on the message stream. Two such generators are used: a small message-only model for the interpretability study, and a larger book-conditioned model that post-training sets out to improve. They share an architecture, a tokenisation and a training recipe, and differ in width, depth and input. This section specifies both and the recipe that produced them.

4.3.1 Model Architecture

Both generators are SSMs built from Mamba-3 layers [22] in the arrangement of Figure 2.2. A message encoder embeds the 26-token stream and passes it through two layers, a fused stack carries the representation onward, and a linear head over the 2,112-id vocabulary reads out the next-token distribution. Each layer normalises its input, applies the Mamba-3 mixer, passes the result through a GELU and a sigmoid gate computed from the same activation, and adds it to the input. Inside the mixer the width is expanded twofold and split into heads of 64 channels, each holding a 64×128 recurrent state, with the rotary phase applied to half of the state dimension.

The dual-stream generator adds a book encoder. After every message the book is summarised as a vector of 503 values: the change in mid-price since the previous message, the time of day and the sub-second part of the timestamp, followed by the resting volume in each of 500 price bins one tick wide and centred on the mid-price, 250 a side, filled from the ten best levels on each side, with bid volumes positive and ask volumes negative. One layer reads this vector at its native width, a dense map brings it to the model width and a second layer follows; the two streams are then concatenated and projected back to the model width before the fused stack. The 503 values are repeated across the 26 positions of the message they describe, so the book enters once per message while the tokens advance one at a time. The message-only generator omits the encoder and the concatenation, and its fused stack is deeper.

Table 4.3 gives the two configurations. The message-only generator has 8.2M parameters at width 256 and 13 layers, each holding a recurrent state of $8 \times 64 \times 128 = 65{,}536$ values; that state is what Chapter 5 probes and steers, and the model is kept small so that it can be read out at every layer over thousands of windows. The dual-stream generator has 78.5M parameters at width 1,024 and ten layers, with 32 heads and a state of 262,144 values per layer; it is the anchor of Chapter 6 and generates about 50k tokens per second on a single GH200 GPU. In line with the data-hungry scaling of Section 2.2.2, the anchor sees about 610 tokens per parameter. That budget represents less than one epoch over the GOOG restriction, which is what leaves the message-disjoint half that post-training draws on (Section 4.1.2).

Table 4.3: The two pre-trained generators. Both use heads of 64 channels with a state dimension of 128 and are trained by the recipe of Section 4.3.2; the dual-stream model adds the book encoder of Figure 2.2.

	Message-only	Dual-stream
Parameters	8.2M	78.5M
Width d_{model}	256	1,024
Mamba-3 layers	13: 2 message, 11 fused	10: 2 message, 2 book, 6 fused
Heads per layer	8	32
Recurrent state per layer	$8 \times 64 \times 128$	$32 \times 64 \times 128$
Inputs	messages	messages, book (503 values)
Corpus	MegaCap-8	GOOG
Used in	Chapter 5	Chapter 6

4.3.2 Pre-Training Recipe

Both generators are trained autoregressively under teacher forcing. Each window of 500 messages is presented as its 13,000 tokens, and the loss is the mean cross-entropy of the next token over every position, including the absolute-time digits that Section 4.2 keeps in the loss. Windows start at a random offset within each day, and where the book is used its vectors are those the real stream induced, so no generated message enters pre-training.

Optimisation follows one rule for both models. Every two-dimensional kernel, the projections inside the mixer, the dense maps and the output head, is updated with Muon [93] at a learning rate of 0.01 and weight decay 0.005. The mixer's recurrence parameters, its step and projection biases and its skip scale, use Adam [94] without weight decay at a rate scaled inversely with width, $5 \times 10^{-4} \times 1024/d_{\text{model}}$, which gives 5×10^{-4} for the dual-stream model and 2×10^{-3} for the message-only one. Embeddings and normalisation scales use AdamW [95] at that same rate with weight decay 0.005. Every rate warms up linearly over the first 1% of steps and follows a cosine to 5% of its peak at the final step, gradients are clipped to unit norm, and no dropout is applied.

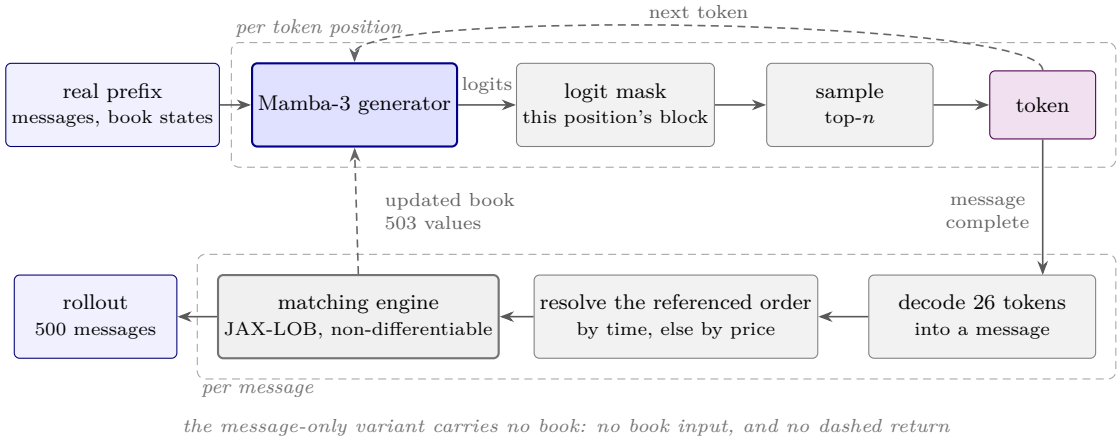


Figure 4.2: The inference loop. Solid arrows carry data, dashed arrows carry state fed back into the model. Each token is drawn from logits masked to the field admissible at that position and becomes the next input. Once all 26 positions are filled the block is decoded into a message, the resting order it references is resolved against the live book, and the true matching engine applies it. The book is held constant across the next message's 26 positions.

Appendix C collects these settings, and those of every other stage, in one table. The dual-stream generator trains for 28,730 steps at a global batch of 128 windows, about 48 billion tokens in roughly 170 GPU-hours; the message-only generator for 30,180 steps at a global batch of 160, about 63 billion tokens. Neither completes a pass over its data, so no window is seen twice. Seeds, hardware and the parallelism behind the global batch are given with the training environment at the end of this chapter.

4.4 Inference Pipeline

Generation runs the pre-trained generator in closed loop with a matching engine: the model emits one message at a time, the engine executes it against a live book, and the book it returns feeds the next message (Figure 4.2).

A rollout starts from a real prefix of 500 messages, passed through the model in recurrent form so that its state summarises the real history, while the engine's book is initialised from a snapshot of the real book at the same instant, 500 levels a side for Chapter 6 and the ten levels of the LOBSTER record for Chapter 5. From

there 500 messages are generated, one token at a time. At each position the logits are restricted to the admissible tokens (Section 4.2) and one is drawn from the 50 most probable in proportion to its score. Positions 11 to 15 are not sampled: the absolute timestamp is computed as the previous time plus the generated interval, re-encoded, and passed through the model like any other input.

A completed block is decoded into a LOBSTER message, with prices read relative to the engine's current mid-price. Since the order ID is never tokenised, a cancellation or deletion is resolved against the live book: first as the resting order on that side whose submission time matches the generated reference exactly, otherwise as the order at the message's price with the closest submission time. The engine applies the message under price–time priority and returns its new mid-price, the time and its ten best levels a side, which are binned into the 503-value vector of Section 4.3.1 and held fixed across the next message's 26 positions.

The message-only generator runs the same loop through a wrapper that accepts the returned book and discards it, so the engine still executes every message and evaluation still reads the book states it produces, while the model's predictions depend on the message stream alone. In both variants logit masking guarantees a well-formed message, not an executable one: a cancellation may name an order absent from the book. Such semantically invalid messages are not corrected. They are executed if the engine allows it and count towards the horizon.

4.5 Evaluation Protocol

The unit of evaluation is a window of 500 consecutive real messages. For probing, the window stands alone: Chapter 5 reads the model's state at a fixed boundary near its end and fits its probes under day-grouped cross-validation (Section 5.1). For realism, the window is the context of Section 4.4 and is extended by the 500 real messages that follow it, so that each window yields one real and one generated continuation from the same starting point. Windows are laid on a fixed grid at a

stride of 1,000 messages from the start of each trading day, so no two overlap, and a panel is drawn from that grid at random without replacement under a fixed seed, across every day it covers.

When several models are compared, all of them are run in a single generation programme. The programme draws the windows once, assigns each window its own random key for sampling, and passes identical windows, keys, engine snapshots and real continuations to every model row, the anchor, every post-trained seed and every control alike. Differences between rows are therefore paired window by window, which separates the effect of the model from the effect of the draw, and the paired bootstrap of Chapter 6 forms its intervals on those differences. A copy of the anchor routed through the same code path as the post-trained rows must reproduce the anchor's rollouts bit for bit, a gate that guards the pairing against numerical drift between rows.

What is computed on the continuations, the LOB-Bench aggregates and position-wise divergence of Chapter 6 together with their selection rule and confidence intervals, is defined where it is used (Section 6.1).

4.6 Training Environment

All computation ran on Isambard-AI [96], whose nodes each carry four NVIDIA GH200 Grace Hopper superchips. The models, the training loop and the matching engine are written in JAX [97] with Flax and Optax, building on the LOBS5 [1] and JAX-LOB [21] code bases, and the chunked state-space scan of Mamba-3 runs in pure JAX without custom kernels. Parameters are held in single precision, with matrix products on the TF32 tensor-core path that JAX selects by default. Our source code can be found at `github.com/alexandre-bismuth/WGAN-Post-Training`.

The dual-stream generator was pre-trained on eight nodes. Each GPU processes four windows per step and gradients are averaged within a node at every step, while across nodes the parameters are averaged once every ten steps; the global batch

of 128 windows of Section 4.3.2 is the sum over all 32 GPUs. The message-only generator followed the same scheme on four nodes at ten windows per GPU. Post-training occupies two nodes per seed, generating 2,048 rollouts of 500 messages that are used by the Wasserstein critic network for realism scoring.

5

Understanding Foundation Models via Mechanistic Interpretability

Contents

5.1 Mechanistic Interpretability Pipeline 46
5.1.1 Model and Data 46
5.1.2 Probing 46
5.1.3 Steering 47
5.1.4 Forward Prediction 48
5.1.5 Robustness 49
5.2 Encoding of Market Statistics 50
5.2.1 Residual Stream vs. Recurrent State 50
5.2.2 Order-Flow Imbalance in the Recurrent State 51
5.2.3 Volume, Volatility, and Depth 52
5.2.4 Per-Layer Encoding Variations 53
5.2.5 Relevance of Specific Messages 54
5.3 Forward Predictions 55
5.3.1 Causal Steering 55
5.3.2 Future-Time Decoding 57
5.3.3 Directional Accuracy 58
5.4 Learning Dynamics 59
5.4.1 Pre-Training Behaviour 59
5.4.2 Generalisation Across Stocks 59
5.5 Summary 61

This chapter takes up the first question of the thesis: what a foundation model of order flow represents once it has been pre-trained by next-message likelihood alone. We read the recurrent state of the message-only generator of Section 4.3 with linear probes, intervene on it by steering, and follow the representation across layers, training and stocks. Section 5.1 specifies the probing, steering and forward-prediction protocols and their controls; Section 5.2 establishes what the state encodes and where; Section 5.3 tests whether the model uses that representation and whether it carries the near future; and Section 5.4 asks when the representation arises during training and whether it transfers to stocks never seen.

5.1 Mechanistic Interpretability Pipeline

5.1.1 Model and Data

The subject of this chapter is the message-only generator of Section 4.3: an 8.2M-parameter Mamba-3 SSM of width 256 and 13 layers, pre-trained autoregressively on the tokenised message stream of the MegaCap-8 corpus (Table 4.3). We analyse its checkpoint at step 30,180. The book is withheld from its input by design (prices enter only as tick offsets from the pre-message mid), so that any book statistic recovered from its state has been reconstructed from order flow rather than read from the input, and the model is kept small so that its state can be read out at every layer over thousands of windows. Each SSM block maintains a fixed-size recurrent state of 8 heads, each a 64×128 matrix, 65,536 values per layer, updated once per token. Probing uses the held-out GOOG panel of January 2026, $N = 2{,}014$ windows spanning 20 trading days; the generalisation experiments extend the same panel to the seven remaining corpus stocks and to JPM and ORCL, two Out-Of-Distribution (OOD) stocks (Section 4.1.2).

5.1.2 Probing

We probe various market metrics (Section 2.4.1) in (a) the per-layer residual stream (256-dim) and (b) the per-layer recurrent state `ssm_state` (flattened 65,536-dim), each at a fixed message boundary near the end of the window (Figure 5.1).

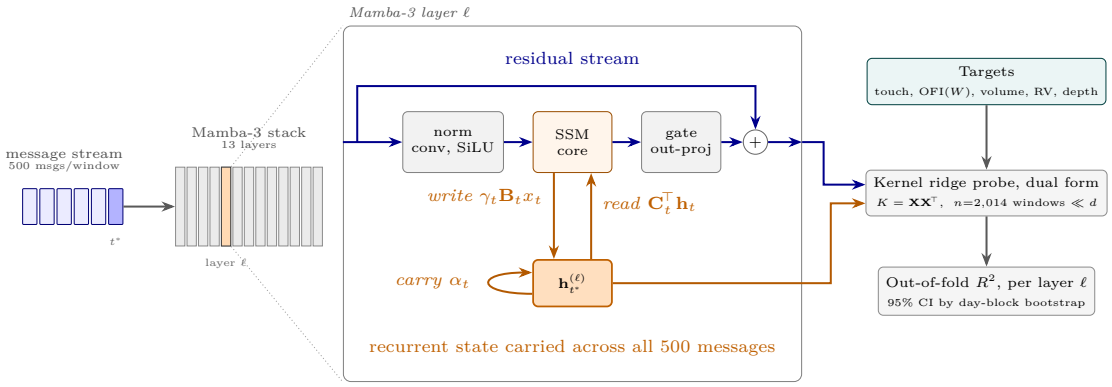


Figure 5.1: Probing loci in a message-only Mamba-3 model. From each layer we extract the per-token residual stream and the recurrent state $\mathbf{h}_t^{(\ell)}$, which accumulates across all messages of the window. Both are read out by the same kernel ridge probe.

Beyond what the state contains, we quantify each past message's contribution to the recurrent state. The Mamba-3 recurrence has an equivalent attention form [22, 23], so the recurrent state at the boundary is exactly a weighted sum of one write per token; summing a message's 26 writes gives its contribution. We recover those weights from the model's own gates and verify on every window that they reproduce its state and layer output up to accumulated floating-point rounding. For each message we then record how much it writes into the state, and how much of what the OFI decoder reads out comes from it.

5.1.3 Steering

A probe can establish only that a quantity is *present* in the state, not that the model *uses* it [36]. Causal steering (Section 2.4.2) writes a belief into the recurrent state and reads out its consequences [37, 38]. The direction $\mathbf{v}$ we write is, for each steered feature (OFI, traded volume, realised volatility), the mean recurrent state of that feature's top-tercile windows minus that of its bottom-tercile ones, added in all 13 layers to the state the model has built from the window, at the observation boundary. One dose (α=1) is exactly that full high–low step (for OFI, from sell- to buy-side pressure), and each feature's dose grid spans the 1st–99th

percentile of where real windows sit along its direction. Effects are judged against 30 scrambled-label axes at matched per-layer norm and an isotropic random direction.

The primary readout is the model's own next-token distribution: after the injection we teacher-force the real continuation and take the difference of the log-probabilities it assigns to the feature-consistent property of the next message (its side, its event type, or the width of its price distribution) against an unperturbed pass over the identical prefix, so the sampling noise does not enter and cross-window market variance cancels window by window. A second readout goes all the way to price: the continuation is generated freely and replayed through the matching engine.

In the market, an order-flow imbalance moves the price through the orders themselves: they arrive at the book, consume liquidity, and shift the quotes. Neither readout reproduces that chain in full. The next-token readout holds the continuation fixed by teacher forcing: the executions never happen, and what we measure is the shift in the model's expectation of them. The engine replay lets the model generate its own messages instead, so executions do occur, but no real orders arrive and the price moves only through what the model chooses to emit; the effect is indirect, and it carries the sampling and replay variance that pairing removes from the next-token readout. We thus treat the next-token distribution as the primary causal measurement and engine replay as corroboration.

5.1.4 Forward Prediction

Beyond what the state encodes about the present and how the model uses it, we also investigate whether this information is useful to predict the future. Future-time decoding uses the same probe on longer windows: 700 messages tapped at the same boundary, so every window carries at least 200 real messages of tail. The targets are the realised mid-price move and the realised level-1 and top-5 OFI over the next H messages, read from the book rather than from a model rollout, so the state is consumed once per window.

Directional accuracy replaces the regression by a one-hot kernel-ridge classifier on the same folds, using 1,500-message windows so that horizons out to H=1000 stay in-window: three classes (down, flat, up), with flat the windows whose mid does not move, and a binary up-versus-down variant restricted to the windows that do. We report accuracy and macro-F1 against the majority-class baseline, since the class balance shifts markedly with H.

5.1.5 Robustness

Overfitting

With 65,536 features per layer and only 2,014 evaluation windows, the obvious objection is that a linear probe of this state can interpolate its labels, and that a reported R^2 measures the fit rather than the model. Four elements of the protocol answer it. Every R^2 is computed out of fold, where interpolation is penalised rather than rewarded, so the reported values are if anything conservative. The random-weights twin shares the full dimensionality and the same windows, and decodes OFI at $R^2 \approx 0.01$, which bounds what the imbalance between features and samples can produce on its own. Under the cross-validated ridge penalty the probe's effective degrees of freedom are about 1% of the state dimensions, so the fitted map is far smaller than the space it lives in. A Principal Component Analysis (PCA) of the state also shows the signal to be low-dimensional: the identical probe refit on the top principal components recovers its full R^2 with 64 components for OFI and 128 for the touch. The probes are accordingly solved in dual form as a kernel ridge regression with an $n \times n$ Gram system, with the layer and λ chosen jointly by nested inner Cross-Validation (CV) on the training days only.

A second concern is estimator variance rather than bias. The reported R^2 is itself a random variable under resampling of trading days, so an unbiased point estimate can still move materially between samples. Held-out R^2 is therefore computed under day-grouped CV, repeated under forward-chaining (expanding-window) CV for the time-series-sensitive targets, and reported with 95% Confidence Intervals (CIs) from

a day-block bootstrap that resamples whole trading days rather than individual windows, so intra-day correlation is preserved.

Controls

On top of that, we use two complementary controls. (i) A random-weights twin with identical architecture separates learned representations from architectural memory or random feature expansion. (ii) Book-snapshot baselines measure what is recoverable from the raw book without any model: linear and Radial Basis Function (RBF) ridge readouts of 28 snapshot features of the L10 book (price offsets, spread, imbalances, microprice, per-level depths) taken at the window start and at the observation boundary, and of the pair augmented with the window's mid move and duration. Since the model never receives the book, state decodability above a single-snapshot baseline certifies reconstruction from messages alone.

5.2 Encoding of Market Statistics

5.2.1 Residual Stream vs. Recurrent State

In Transformer-based foundation models, mechanistic interpretability probes the residual stream. In our SSM that locus is nearly empty of market structure, and the representation sits in the recurrent state instead (Figure 5.2). Linear probes on the residual stream recover only a weak, slow price-level signal and no integrative market quantities, with order-flow imbalance, traded volume, and realised volatility all decoding at $R^2 \leq 0.06$ from the residual stream at $W{=}500$, against 0.50–0.61 from the recurrent state. A graded lag-k control (decode the message k steps back) explains why: the residual stream's linear memory horizon is only $\sim$2–5 messages, so quantities that must be integrated over hundreds of messages cannot live there.

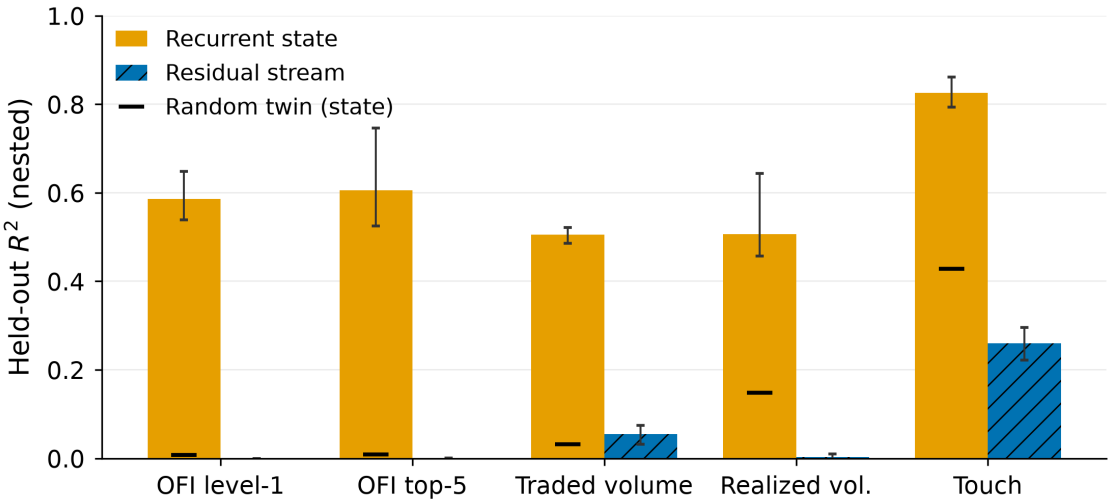


Figure 5.2: Held-out R^2 (layer chosen by nested CV) decoded from the recurrent state vs. the residual stream (W=500; same windows, probe, and CV; whiskers: 95% CIs). Market statistics live only in the recurrent state; touch is partially readable everywhere but sits on a high architectural floor.

On the other hand, the recurrent state linearly encodes market microstructure. Selecting the layer by nested CV, we decode statistics absent from the residual stream. Top-of-book (touch) reaches $R^2 \approx 0.83$ in the SSM state compared to only $R^2 = 0.26$ in the residual stream. Comparing R^2 performance with a random-weights twin, we find that learning yields a 93% improvement ($0.43 \rightarrow 0.83$), the twin's high floor reflecting that recent messages partially echo the current quotes even through random recurrent features. On top of that, a readout of the full, stale book snapshot from $\sim$500 messages earlier reaches only $R^2 = 0.49$, indicating that the state tracks the live touch from messages alone rather than simply reconstructing the book.

5.2.2 Order-Flow Imbalance in the Recurrent State

Our foundation model successfully learns and encodes order-flow imbalance in its recurrent state (Table 5.1). We make three observations. First, our model encodes flow rather than snapshots, OFI decoding $\sim$5$\times$ better than a readout of the full boundary book snapshot (0.59 vs. 0.12). Second, unlike touch, the random-weights twin scores ≈ 0 for OFI at W=500, so training is the only source of performance

there, and a label-permuted probe recovers $R^2 = -0.003$, so the fit is selective to the target rather than to the state's dimensionality. Third, OFI results survive forward-chaining time-series cross-validation at every lookback, with the largest relative margin over the snapshot at W=10 (0.29 vs. 0.02).

Table 5.1: Held-out OFI R^2 (layer chosen by nested CV) from the trained SSM state, compared with a random-weights twin and the best readout, chosen by nested CV, of the full book snapshot at the observation boundary (Book), given a lookback window W.

Target	Trained	fwd-CV	Random	Book
OFI_1 W=500	0.59	0.53	0.01	0.12
OFI_1 W=100	0.49	0.44	0.03	0.12
OFI_1 W=10	0.29	0.26	0.07	0.02
OFI_5 W=500	0.61	0.51	0.01	0.08

Given the role of OFI as a driver of short-horizon price moves [19] and a basis of the deep prediction literature [20], the encoding of such flow statistics suggests that LOB state-space foundation models carry the kind of flow information downstream predictors rely on; we test its forward value in Section 5.3.

5.2.3 Volume, Volatility, and Depth

Decoding is not limited to the touch and OFI: the recurrent state also decodes traded volume (the log total size of visible executions over the window), realised volatility (the standard deviation of per-message mid-price changes over the window, capturing how much prices actually moved), and book depth (the log-transformed cumulative resting volume across the first ten price levels on each side of the book, a proxy for available liquidity) under both CV protocols (Table 5.2). Traded volume and realised volatility decode far above both the random-twin floor and the book-snapshot baseline. Depth is the exception: a stale snapshot from 500 messages earlier predicts boundary depth better (0.64) than the state does (0.38); resting liquidity is highly persistent, and the message-built reconstruction retains only a coarse summary of it.

Table 5.2: Held-out R^2 from the SSM state (layer chosen by nested CV) vs. the random-weights twin and the best book-snapshot baseline, chosen by nested CV (Book), for a lookback window W and including forward-chaining CV. Brackets are 95% day-block bootstrap CIs. For depth, Book is the stale window-start snapshot, since the boundary snapshot contains the target itself.

Target ($W = 500$)	Trained	fwd-CV	Random	Book
Traded volume	0.50 [0.49, 0.52]	0.44 [0.40, 0.46]	0.03	0.11
Realised volatility	0.51 [0.46, 0.64]	0.50 [0.42, 0.62]	0.15	0.23
Log-vol L10 book depth	0.38 [0.32, 0.43]	0.30 [0.23, 0.36]	0.09	0.64

5.2.4 Per-Layer Encoding Variations

Decodability is far from uniform across the stack (Figure 5.3). OFI_1 (W=500) peaks in the early fused trunk ($R^2 = 0.59$ at SSM layer 3), falls to near zero through the middle of the stack, and partially recovers at the penultimate layer (0.16); touch follows the same shape at a higher level, recovering to 0.62.

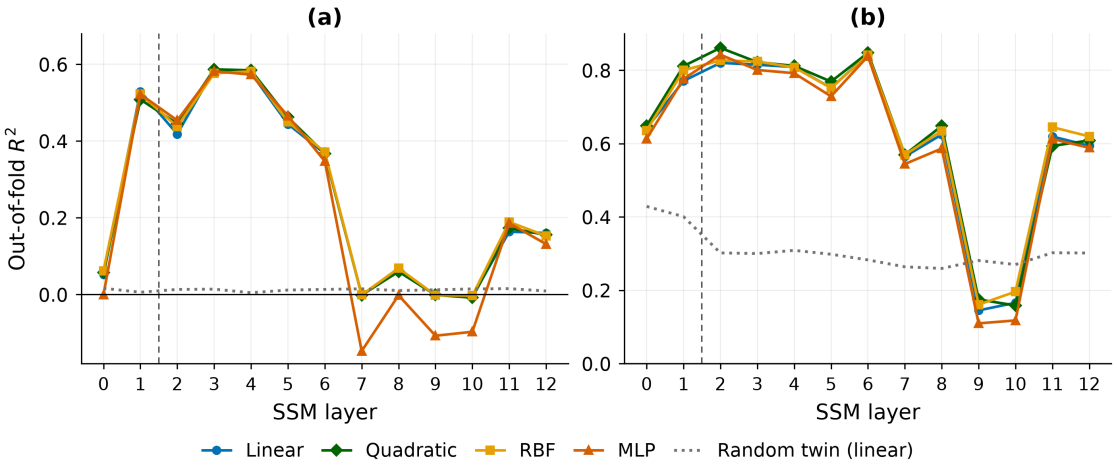


Figure 5.3: Per-layer decodability for (a) OFI_1 (W=500) and (b) touch (best-ask offset). We test four probes of increasing capacity: linear, quadratic, RBF kernel ridge, and a one-hidden-layer Multilayer Perceptron (MLP). All four curves concur, so the mid-stack dip is not a non-linear re-encoding but genuine absence.

The dip is not a non-linear re-encoding. Quadratic, RBF-kernel and one-hidden-layer MLP probes follow the linear curve at every layer, so the mid-stack states do not hold OFI in a form a linear readout misses. At the dip layers the trained state scores at or below its random-weights twin.

5.2.5 Relevance of Specific Messages

Probes establish what the state encodes, but not how that code is assembled from the message history. In a message-only model the recurrent state is the entire memory, so this is a practical question: how much history is actually in play, and how long does a single event keep mattering? Mamba-3's SSD makes the answer exact rather than inferred. The state at the observation boundary is a weighted sum of per-token writes, so summing them within each message attributes it message by message, a recurrent analogue of an attention map, as described in Section 5.1. We trace two quantities back to the messages that wrote them, the state as a whole and the single direction the OFI probe reads out.

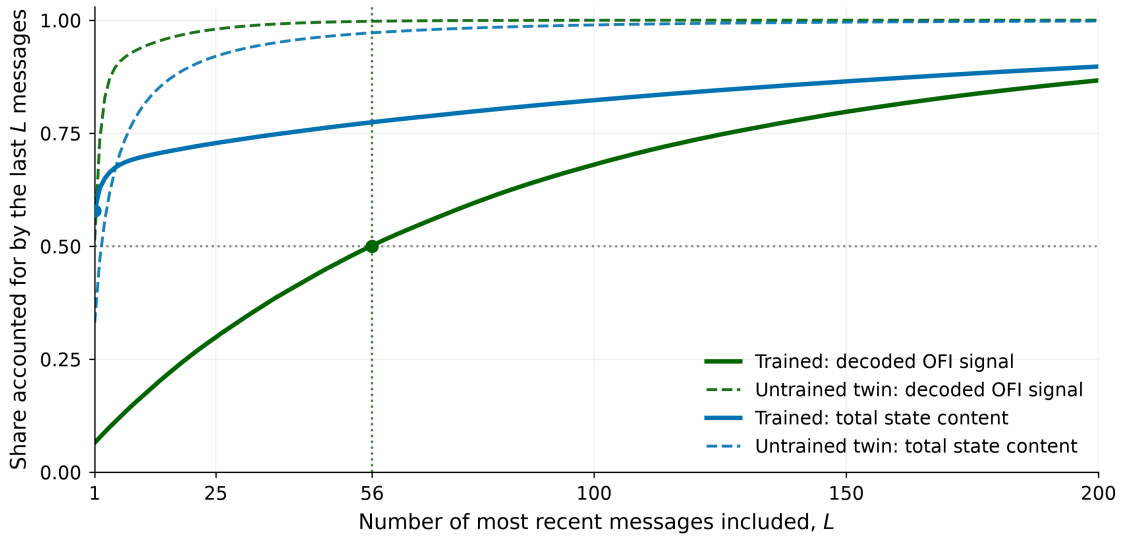


Figure 5.4: Share of the recurrent state (blue; all layers) and of the decoded OFI signal (green; best layer) accounted for by the L most recent messages, for the trained model and its random-weights twin. Half of the raw state is the newest message alone; half of the OFI signal takes the last 56 messages.

The two curves come apart (Figure 5.4). The single newest message accounts for 58% of the total write magnitude, and the last 50 messages for 77%, so a state snapshot is mostly a description of the present. The decoded OFI signal behaves in the opposite way: the newest message contributes 7%, and half the signal needs the last 56 messages. Order flow is therefore not held in the recency mass but in a small long-memory subspace. Of the eight heads at the probe layer, four have

state half-lives of 28–66 messages while the other four forget within a message. In the random-weights twin both curves collapse onto the newest few messages (51% of the readout from one message, 89% within five; no head at that layer exceeds a 10-message half-life), so the long-horizon integrators are built by training rather than given by the architecture. The decoded order flow is an integrated regime variable rather than a running per-event sum, since individual message contributions correlate with their own realised OFI at only $r \approx 0.11$.

5.3 Forward Predictions

Beyond market simulators, we evaluate our model's capacity to respond causally to a change of conditions and to represent the market ahead of time. These experiments test whether the representation can be used: whether interventions on the state move the model's predictions, and whether the state carries signal about the market's near future.

5.3.1 Causal Steering

The most direct readout of a causal change in a model trained autoregressively is through the next-token distribution. Following a difference-of-means approach (Section 5.1.3), we push the recurrent state at the last context message along each encoded feature's axis, simulating high or low flow (OFI), heavy or light trading (traded volume), and turbulent or calm prices (realised volatility), then teacher-force the real continuation, and observe per-message log-probabilities. Results are compared against a shared α=0 pass, where α represents one top-vs-bottom tercile displacement.

Table 5.3: Model's beliefs about the next message after pushing each feature's state axis by $\pm\alpha$ using the top-vs-bottom tercile difference-of-means steering approach. The shuffled-label null rows give the mean response of 30 norm-matched axes built from shuffled labels. Perplexity measures the effective width in one-tick levels of the next-price forecast.

Steered axis	Quantity	$-\alpha$	0	$+\alpha$
Order flow (OFI)	$\mathbb{P}(\text{side}_{t+1} = \text{buy})$	0.45	0.51	0.57
	shuffled-label null	0.51	0.51	0.51
Traded volume	$\mathbb{P}(\text{type}_{t+1} = \text{exec})$	0.005	0.013	0.034
	shuffled-label null	0.013	0.013	0.013
Realised volatility	$\text{Perplexity}(\text{price}_{t+1})$	5.2	8.2	19.0
	shuffled-label null	8.8	8.2	9.4

The model's beliefs respond cleanly for all three features (Table 5.3), symmetrically for order flow and traded volume: a volume push scales $\mathbb{P}(\text{type}_{t+1}{=}\text{exec})$ by $\sim$2.7$\times$ in either direction at these doses, while the volatility response more than doubles the width of the next-price forecast under a turbulence push and sharpens it under a calm push, saturating at large calm doses (most windows are already quiet). None of the 30 shuffled-label axes reaches the trained response: the responses follow the specific encoded directions, not the size of the push.

Steered beliefs only partially materialise once the generated messages hit the book. We roll the steered state out for 50 messages through the matching engine and compare each realised statistic against the shuffled-label null. The volume push carries through to the market: median executed volume rises from 25 shares at $\alpha{=}0$ to 187 at the +dose and falls to 1 at the $-$dose, while the mid and the generated order flow stay at null. The flow push carries through to the order flow but not to the price: the generated order flow shifts in the pushed direction, but the mid stays within the null band, with only a weak sell-side drift. The volatility push does not carry through: realised volatility stays within the null band. The model expects prices to disperse, yet does not generate more volatile paths, even though the volume push does raise realised volatility, likely because its extra executions consume depth. Causal control over the generated market thus ranks traded volume > order flow >

volatility, ordered by how directly single messages determine the statistic: equally decodable features (Tables 5.1 and 5.2) are not equally controllable.

5.3.2 Future-Time Decoding

Steering aside, the recurrent state frozen at the last message of the context window can also be read directly for future market information, without any autoregressive rollout, and compared with a random-weights twin.

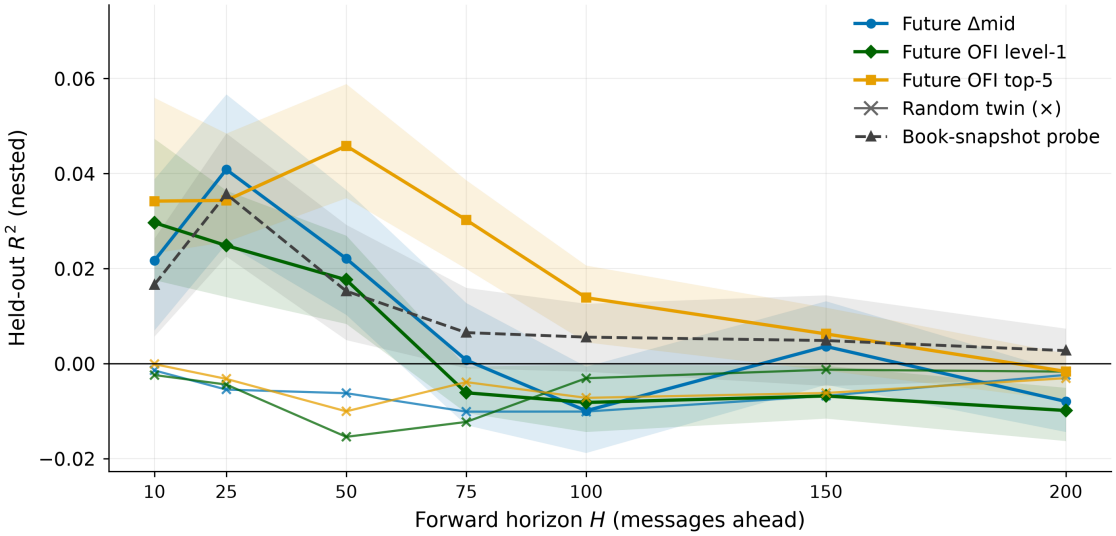


Figure 5.5: Future-time decoding: held-out R^2 of linear state probes for future Δmid and OFI vs. horizon H (bands: day-block 95% CIs; ×: random twin). Grey dashed: book-snapshot probe for future Δmid from the raw L10 book; its future-OFI R^2 stays below 0.02 (not shown).

A positive held-out R^2 indicates a predictive signal. The state carries only a weak one (Figure 5.5): it explains 4% of the variance of the next-25-message mid-price move and at most 5% of future top-5 OFI, with no signal left beyond $H{\approx}100$. This is level with a probe handed the L10 book snapshot instead of the state ($R^2 = 0.036$ at $H{=}25$), while the random-weights twin stays at or below zero at every horizon. The representation thus tracks the current flow but adds little forward information beyond what the visible book already holds, in line with the weak short-horizon predictability of mid-price moves.

5.3.3 Directional Accuracy

We also test forward prediction as three-class directional accuracy at several horizons (Figure 5.6).

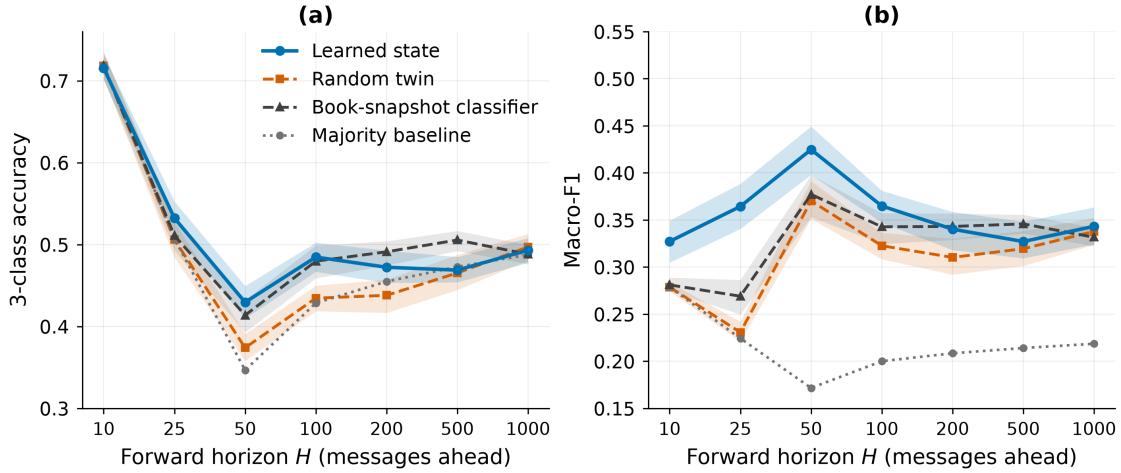


Figure 5.6: Three-class (down/flat/up) mid-price direction from a linear decoding of the recurrent state (W=500) vs. forward horizon H: (a) accuracy (majority-inflated at short H), (b) macro-F1 (true three-class skill). Grey dashed: book-snapshot baseline classifier using the raw book as input.

At H=50, the linear probe of the frozen state reaches 42.9% three-class accuracy (macro-F1 0.425), against 34.7% for the majority class, 37.4% for the random-weights twin, and 41.4% for the book-snapshot baseline.

Three-class accuracy can nonetheless be misleading, as over short horizons a large part of it comes from calling the flat class correctly. Another interesting angle is thus to test for directional accuracy conditioned on a mid-price move (i.e. binary directional accuracy). In this case, the same probe is right 61.2% of the time at H=25 and 60.0% at H=50, against 50.5% for the majority baseline and 50–51% for the random-weights twin, but level with the book-snapshot readout (61.8% and 60.9%). Skill fades with horizon, to 57.8% at H=100 and 53.0% at H=200, and reaches chance by H=500.

5.4 Learning Dynamics

5.4.1 Pre-Training Behaviour

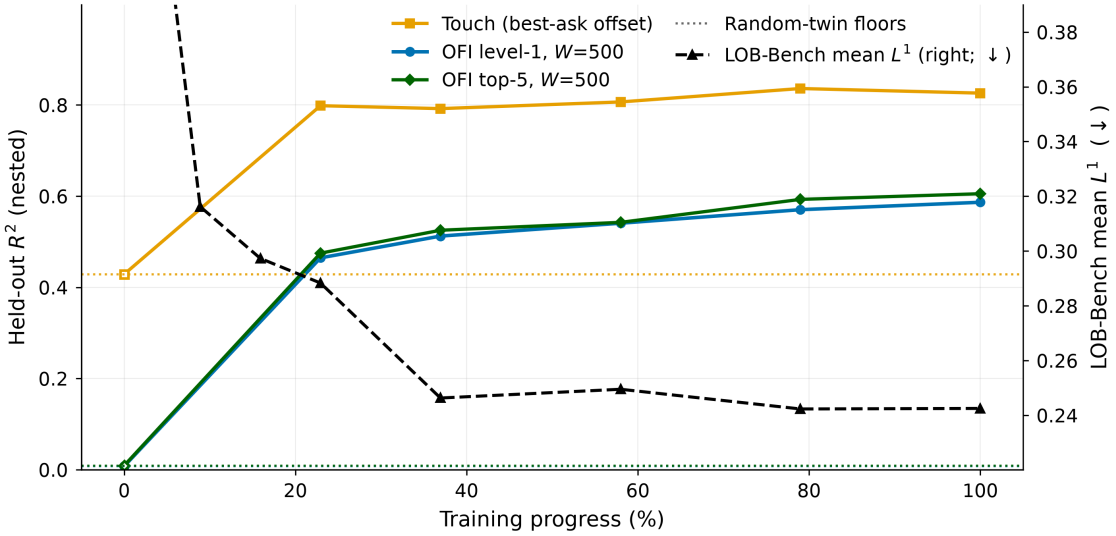


Figure 5.7: Market statistics decodability and LOB-Bench scores at different parts of training (0% = random-weights twin). Touch is learned early, OFI keeps improving over the whole run, and LOB-Bench plateaus after about a third of training.

Touch is a market feature learned early in training (Figure 5.7). While linear decoding of the touch plateaus quickly, OFI's decoding improves steadily throughout training, without a clear plateau. This reinforces our belief that our message-only foundation model learns to represent the market as a flow and not as a static book snapshot, and suggests that despite cross-entropy and LOB-Bench hitting a performance plateau, longer pre-training may reinforce OFI encoding within the recurrent state.

5.4.2 Generalisation Across Stocks

The recurrent state linearly encodes the top-of-book across in-training and out-of-distribution stocks (Figure 5.8), suggesting that the models learn a general representation of those market features beyond per-stock characteristics, in line with the universal structure of price formation reported across instruments [56]. However, realised order-flow imbalance R^2 performance varies across stocks.

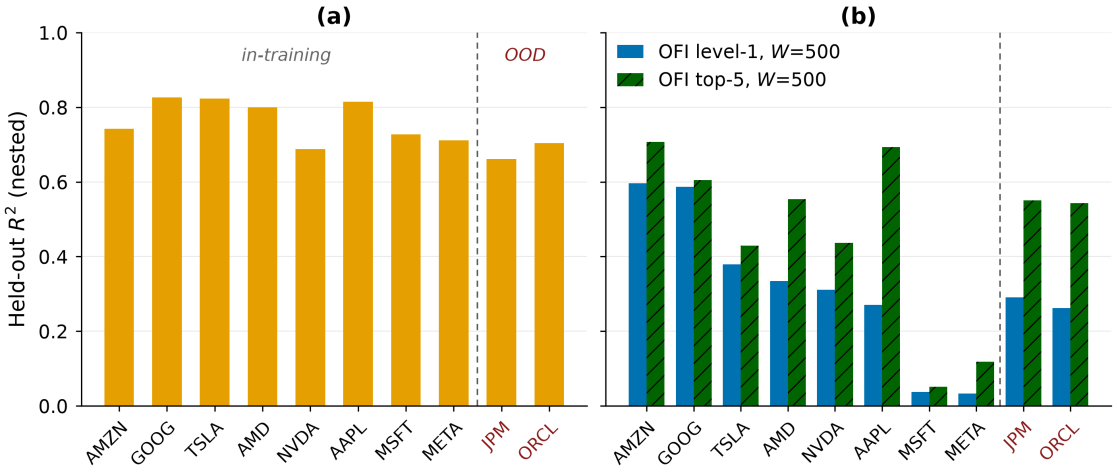


Figure 5.8: Held-out R^2 (layer chosen by nested CV) of probes on the frozen SSM state for ten stocks and W=500. (a) Touch decodes uniformly, including on out-of-distribution stocks, while (b) OFI R^2 varies widely.

OFI out-of-fold R^2 decreases linearly with the share s of target variance held by the top-1% of windows (Figure 5.9). This relation highlights the current struggle of LOB foundation models with distribution tails: the probe explains none of the extreme-burst variance, yet decodes ∼80% of typical-window OFI variance on average. To mitigate the effects of tails and recover linear decoding performance, several methods exist. One option is to winsorise OFI, capping the top and bottom 1% of episodes at the 1st and 99th percentile values. Another option is to use Rank R^2, which replaces each episode's OFI value by its normalised rank among all evaluation windows. The resulting R^2 rewards recovering the ordering of episodes rather than their magnitude, and is therefore insensitive to heavy tails by construction. Under either metric, OFI is represented across the board for OFI_1 ($R^2 \geq 0.4$) and OFI_5 ($R^2 \geq 0.5$) for both in-training and out-of-distribution stocks.

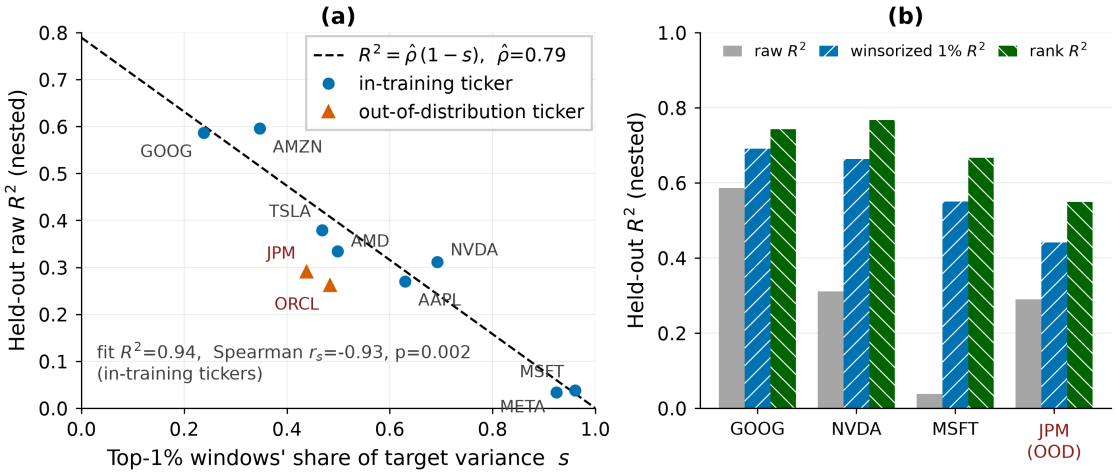


Figure 5.9: (a) OFI_1 R^2 versus the top-1% windows' share s of target variance. Tail prevalence explains variations; OOD stocks fall close to the same line. (b) Raw vs. winsorised-1% vs. rank-transform R^2 for a representative subset: both tail-robust metrics restore performance to a common band.

5.5 Summary

This chapter answers the first question of the thesis. A model trained only to predict the next message holds a representation of the market in its recurrent state as a flow rather than a snapshot: OFI decodes far better from the state than from a single book snapshot, a random-weights twin decodes none of it, and the signal is drawn from a small long-memory subspace. OFI is the slowest thing the model learns, and it transfers to stocks never seen in training. The representation is used, since steering it moves the model's beliefs as each feature's meaning dictates. Its forward information, however, adds little to what the visible book already carries. Chapter 6 turns from what the model represents to what it generates.

6

Adversarial Post-Training with Evolutionary Search

Contents

6.1 Adversarial Pipeline 63
6.1.1 Generator Updates 63
6.1.2 Critic 64
6.1.3 Optimisers 65
6.1.4 Controls 66
6.1.5 Evaluation 66
6.2 Results 67
6.3 Training Dynamics 68
6.3.1 Reward Hacking 68
6.3.2 Loss Behaviour 69
6.4 Compute Efficiency 69
6.4.1 Compute-Optimal Frontier 70
6.4.2 Benefits Over Pre-Training 70
6.5 Critic and Optimiser Ablations 71
6.5.1 Input Features 71
6.5.2 Optimisation Method 72
6.6 What Post-Training Changes 72
6.6.1 Compounding Error 73
6.6.2 Market Realism 74
6.6.3 Market Decoding 76
6.6.4 Downstream Performance 78
6.7 Summary 79

This chapter takes up the second question of the thesis: whether a pre-trained generator can be improved by an objective that no gradient reaches. The objective is adversarial, a Wasserstein critic trained online to separate real windows from free rollouts executed through the matching engine, and the optimisers are zero-order, so that the critic's score is the only signal the generator receives. Section 6.1 specifies the pipeline, its controls and the sealed evaluation; Section 6.2 reports the headline result; Sections 6.3 to 6.5 examine the training dynamics, the rollout budget and the two central design choices; and Section 6.6 measures what the optimisation changes beyond its aggregate score.

6.1 Adversarial Pipeline

At each step, the generator rolls out $G \times Q = 16 \times 128 = 2{,}048$ perturbed continuations through the matching engine, the critic scores each rollout, and the optimiser turns those scores into an update of the generator's interior projection kernels, built from LoRA [52] perturbations. This section specifies the four components in turn, the generator update, the critic, the optimisers and the controls, then the evaluation that every experiment of this chapter shares.

The generator is the anchor of Section 4.3, pre-trained and post-trained on the same GOOG corpus (Section 4.1.2). The restriction to one stock is deliberate: a specialist anchor is the strongest generator available for the stock it is tested on, and the book it receives after every generated message concentrates the adversarial signal on that stock's microstructure. Preliminary runs on a generator pre-trained on the full S&P 500 universe showed smaller gains, whether it was post-trained on a matched multi-stock corpus or on GOOG alone, and are not reported here. The specialist setting is therefore the only one studied in our post-training framework.

6.1.1 Generator Updates

The pre-trained generator is frozen except for the interior projection kernels of the SSM blocks, which EGGROLL perturbs with rank-4 LoRA factors and updates

in full. We intentionally exclude other parts of the SSM architecture to keep ES low-variance while preserving the pre-trained dynamics encoded in the SSM core, embeddings, and decoder head. On top of that, a KL divergence trust region of $\lambda = 0.1$ is applied inside the fitness to prevent reward-hacking drift.

6.1.2 Critic

Wasserstein Objective

The critic is trained from scratch and updated online alongside the generator on the Wasserstein-GAN objective with the R_1 penalty (γ=10), averaged over the K=5 independent ensemble members, using Adam at a learning rate of 3×10^{-4} with β_1=0 [43]. The R_1 term is the zero-centred gradient penalty of Section 2.5.1, applied to real windows only, which flattens the critic around the data manifold and keeps the online adversarial updates locally convergent [44]. Trained in this way, its decision boundary tracks the live discrepancies that free rollouts exhibit, including compounding errors that cross-entropy never penalises.

Critic Input

The critic reads decoded messages rather than summary statistics, so that its representation is learned rather than designed. We perform lossless transforms of the message fields of Section 4.2 into channels (Figure 6.1): four one-hot fields for event type, a direction, a price, a size, a time difference, and four reference channels, defined for messages that reference another order. We intentionally discard absolute price, absolute time, and order IDs, since such trivial tells would separate real from fake without reflecting realism: NA in fake rollouts, non-stationary or pure bookkeeping in real data. These channels are combined with median normalisation to mitigate heavy tails that dominate scales. The pool of 512 real context windows is redrawn every five steps, keeping the critic's real set briefly stationary while preventing memorisation of individual windows. At each step, Q=128 contexts are sampled without replacement and shared across perturbations so fitness circumvents context-sampling noise.

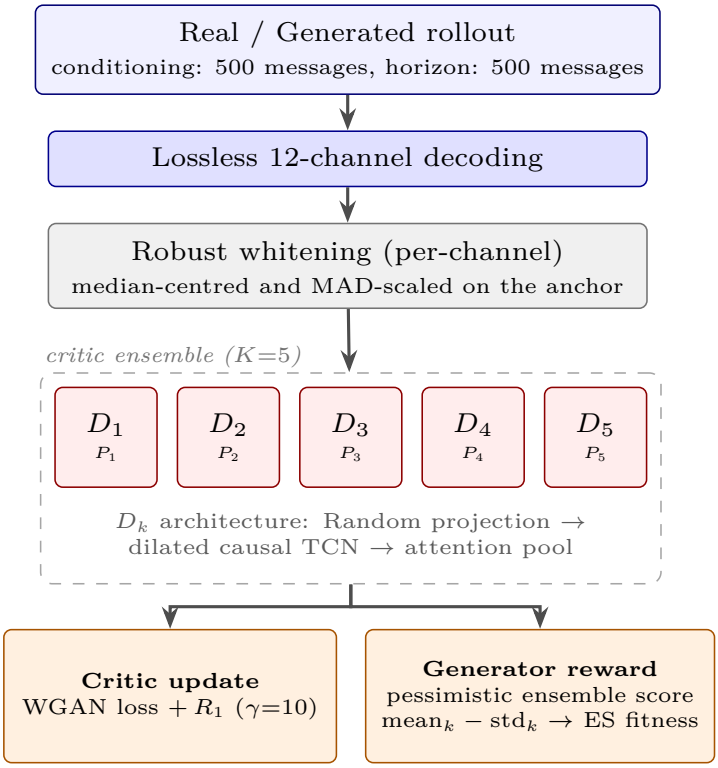


Figure 6.1: Raw-message critic. Each 26-token message is decoded into 12 channels while absolute values and IDs are withheld as trivial tells. Channels are median-centred and scaled by $1.4826 \cdot \text{MAD}$ (median absolute deviation) to match a Gaussian distribution. Spectral-normalised TCN members, each behind its own fixed random projection, score the 500-message continuation and the generator receives the pessimistic ensemble score.

6.1.3 Optimisers

EGGROLL

Since no gradient crosses the matching engine, the generator is updated with EGGROLL [12]: G=16 members (eight antithetic directions) of rank r=4 with σ=0.003 (swept), evaluated on Q=128 shared contexts. A direction's fitness is its pessimistic ensemble score ($\text{mean}_k - \text{std}_k$), penalised by the KL trust region of Section 6.1.1, centred-ranked per context, and averaged over contexts; the update is applied by Adam at 5×10^{-4} learning rate.

GRPO

For comparison, we swap the ES update for GRPO [13] under the same critic and contexts, with per-context groups (G=8), z-scored advantages, a single on-

policy step, and a stronger trust region ($\lambda = 0.3$), as raw advantages lack the rank transform's bounding. Section 6.5.2 reports the comparison.

6.1.4 Controls

Two controls bound what the pipeline could manufacture without the critic's signal. The first is a selection-matched null in which training is replaced by drawing N=8 random LoRA perturbations of the base model by seed, matched to the deployed checkpoints in parameter norm, and passed the identical selection and sealed-test pipeline. It bounds how much apparent improvement the pipeline can manufacture from selection bias. The second is continued pre-training: the anchor is trained further on the same unseen corpus under its original recipe, at a compute budget matched to a single post-training seed (the IsoFLOP control of Table 6.1) and, in a second variant, to the compute of the full experiment with its selection rollouts included. It bounds how much of the gain additional likelihood training on identical data could reproduce.

6.1.5 Evaluation

Benchmark

Rollout realism is measured with LOB-Bench [5], following the protocol of Section 2.3.1, in which real and generated continuations are mapped through the benchmark's fixed set of scoring functions and compared one distribution at a time. We aggregate the benchmark's 21 unconditional scores into two summary numbers: WS-21, the mean over the 21 scores of the Wasserstein-1 distance between real and generated marginals, and Mean-L_1, the mean of the corresponding L_1 histogram distances, both as defined in (2.3). Uncertainty is quantified with a paired window-level block bootstrap. Whole evaluation windows are resampled with replacement (500 replicates), each replicate re-scores real and generated continuations from the same drawn windows, and model-versus-anchor deltas are formed replicate-wise from these shared draws, so window-composition noise cancels in the comparison. Day-clustered resampling over the 28 panel days yields consistent intervals.

Selection

Any learned reward is eventually over-optimised [91], and ours is a live adversary: past a certain point the generator raises its critic score by exploiting the critic rather than by becoming more realistic, so the last checkpoint cannot be the deployed one. A three-seed pilot trained to 50 steps locates that turn (Section 6.3.1), with realism improving monotonically through steps ≈15–20 and then decaying. We therefore train the headline seeds for 30 steps and score a checkpoint grid spaced every two steps. Each is scored offline with WS-21 on 2,048 context windows (1,024 for comparative studies) drawn from the training timeframe, and the best is selected as the winner; the picks land at steps 10–20 (median 16). The criterion uses no critic signal by design, so a checkpoint cannot select itself through the same exploit that inflated its reward.

Test

Selected checkpoints are scored once on 4,096 disjoint windows (2,048 windows for comparative studies) from a 28-trading-day panel of most recent unseen data (January–February 2026). Results are compared against the pre-trained model re-scored on the same windows.

6.2 Results

The optimised pipeline results are shown in Table 6.1: adversarial post-training improves held-out realism for every seed, with WS-21 falling by 17.6% against the base pre-trained model and Mean-L_1 by 17.0%; the paired 95% bootstrap interval excludes zero for the ten-seed mean and for each seed individually (per-seed WS-21 range −11.3% to −23.9%), while both controls end above the anchor.

Table 6.1: Sealed test at 4,096 held-out windows (28-day panel). Δ is the mean change across n=10 seeds relative to the frozen pre-trained model, with paired window-block bootstrap 95% CIs; controls are single rows from the same generation programme.

Model	WS-21	Δ [95% CI]	Mean L_1	Δ [95% CI]
Pre-Trained	0.194	—	0.179	—
Post-Trained	**0.160**	−17.6% [-19.7, -15.3]	**0.149**	−17.0% [-18.9, -15.8]
Selection-null	0.204	+5.3% [+2.5, +8.1]	0.193	+7.8% [+5.9, +9.6]
IsoFLOP ctrl	0.201	+3.5% [+0.5, +6.5]	0.186	+3.9% [+1.9, +5.5]

This adversarial pipeline consumes only about 50 M training tokens, a $10^3\times$ reduction from the ≈48 B-token pre-training corpus. With 12 GPU-hours per seed, our pipeline shows that HFT foundation models can largely benefit from adversarial post-training as an inexpensive refinement stage.

The gains also reach the failure mode that motivates post-training, the compounding of errors over free-running rollouts, which Section 6.6.1 measures position by position together with what else the optimisation changes.

6.3 Training Dynamics

6.3.1 Reward Hacking

The per-seed training dynamics also reveal the need for external selection: with three seeds trained on 50 steps, all seeds improve monotonically to steps ≈15–20, reaching −16.4% to −21.7% against the anchor on the selection windows, then decay as the generator begins to exploit the critic, seed 1 returning past the anchor (+25.4%) by step 50 and seeds 0 and 2 to roughly anchor level. The WS-21 grid selection catches the turn in all three seeds. The softer sealed-test deltas (−17.5% mean, against −19.4% at selection) reflect ordinary selection optimism; the sign is preserved out of sample.

The critic tells the same story from the other side. Its held-out real-versus-fake discrimination starts strong (AUC ≈0.62 at the first step) and declines steadily

to chance level (≈0.51, calibration cross-entropy peaking near 0.80 nats) at steps 20–25, so the generator genuinely closes the gap the critic can measure, exactly where grid selection places the turn. Past the turn the equilibrium destabilises: discrimination recovers to AUC ≈0.64 and the cross-entropy drops below the $\ln 2$ chance level for the only time in training, the online critic cleanly re-separating an adversary whose continued reward ascent has decoupled from realism. The ten 30-step headline seeds retrace the approach leg of the same arc (mean AUC $0.62 \to 0.52$ by step 20).

6.3.2 Loss Behaviour

Adversarial gains cost little likelihood. Teacher-forced next-token cross-entropy on 512 held-out real windows rises by +0.10% (95% CI $[+0.07\%, +0.12\%]$) by step 20 across the ten headline seeds, where it stays through step 30. Nor is the concession a uniform drift: per-seed values at step 30 span -0.38% to $+0.41\%$, while the KL divergence to the anchor grows monotonically in every seed ($2.0 \to 6.2\times10^{-3}$ between steps 10 and 30). The update thus moves steadily away from the pre-trained model while leaving its likelihood almost untouched.

6.4 Compute Efficiency

All of our results are obtained with populations of just G=16 members (eight antithetic direction pairs), five orders of magnitude below the ∼1M-member populations with which EGGROLL was introduced for from-scratch pre-training [12]. The regimes differ fundamentally. Post-training explores the immediate neighbourhood of a strong anchor. The scarce resource is instead the reliability of each direction's score, since market rollouts are a noisy fitness signal. We therefore average each direction's fitness over Q=128 shared contexts, spending the rollout budget on update stability rather than population size. At a fixed budget $G\times Q$, the outcome should then be governed by the total rather than the direction count, and small populations should lose nothing, as Section 6.4.1 confirms experimentally.

6.4.1 Compute-Optimal Frontier

We ablate the split between direction count G and contexts per direction Q at a fixed budget of 2,048 rollouts per step, three seeds per arm, holding everything else constant. On the sealed test, different allocations all sit well inside the within-arm seed spread. The split shows instead at selection, where larger-G arms tend to select earlier checkpoints (median picked step 20 at G=16 versus 15 at G=32 and G=64), since sharper per-step gradient estimates climb the critic's proxy faster and approach the reward-hacking window sooner, while the co-trained critic receives the same number of updates to keep pace.

Restoring Q=128 at G=64 quadruples the budget to 8,192 rollouts per step and buys nothing: its seed lands at -13.5% WS-21 and -15.8% Mean-L_1, below the WS-21 seed range of the fixed-budget arms. Two conclusions follow. At a fixed budget, the sealed outcome is insensitive to how rollouts are split between directions and contexts. Furthermore, growing the population without shrinking the contexts per direction (to more closely resemble ES pre-training) buys no additional sealed realism either: against a critic trained online, the binding constraint is the critic's robustness, not population size.

6.4.2 Benefits Over Pre-Training

The improvement is not a token-budget effect that more pre-training could reproduce. The continued pre-training control of Section 6.1.4 ends slightly worse than the anchor at the training-only match (+3.5% WS-21, Table 6.1), and extended to the compute of the full experiment it lands within noise (+2.4%, paired CI spanning zero). At either budget, additional likelihood training on identical data reproduces none of the adversarial gain, revealing the value of the critic's sequence-level signal beyond data volume or extra compute. The compounding-error analysis of Section 6.6.1 confirms this: extra pre-training does not slow the accumulation of errors over free-running rollouts.

6.5 Critic and Optimiser Ablations

Holding the training loop fixed, we ablate the two central design choices of the pipeline: what the critic observes, and how its scores become generator updates.

6.5.1 Input Features

The raw-message critic of Section 6.1 is the endpoint of an ablation over the critic's input. Reusing the generator itself, scoring rollouts through the frozen backbone's pooled hidden state with a shallow head, appears free but is structurally exploitable: the adversarial reward turns anti-aligned with held-out realism (correlation +0.5–0.8 between reward and LOB-Bench error), and two of three seeds decay back to the anchor. Hand-designed per-step microstructure descriptors, scored by the same spectral-normalised Temporal Convolutional Network (TCN) ensemble, restore a reliable signal but restrict the critic to discrepancies the descriptors can express. The losslessly decoded message stream removes both failure modes, leaving representation learning to the adversarial objective.

Table 6.2 quantifies the comparison under the otherwise-identical protocol: three seeds per arm, $G \times Q = 16 \times 128$, 50 steps, unseen-corpus contexts, identical selection, and one single-programme sealed test in which every row is generated under common random numbers and scored against the same anchor. The raw-message critic roughly doubles the descriptor critic's sealed effect, improving WS-21 by 17.0/15.7/19.7% per seed (Mean-L_1 17.2/18.4/21.1%) against a descriptor-critic mean of 8.5% (6.7%). Every raw-critic seed beats every descriptor-critic seed on both metrics, and the descriptor arm's weakest seed is statistically indistinguishable from the anchor (-2.4% WS-21, paired CI spanning zero). The backbone critic's selected checkpoints score well on average but are the least reliable of the three arms, spanning -11.5% to -28.2% across seeds, consistent with an exploitable critic rather than the reward remaining a faithful realism signal throughout training.

Table 6.2: Critic-input ablation. Single-programme sealed test at 2,048 held-out windows (28-day panel), all rows sharing one anchor; Δ = mean paired change vs. that anchor across three seeds, with the per-seed range (minimum, maximum) in brackets.

Critic input	ΔWS-21 $\downarrow$	ΔMean L_1 $\downarrow$
Generator backbone	-21.4% $[-28.2, -11.5]$	-18.7% $[-23.7, -10.0]$
Microstructure descriptors	-8.5% $[-11.6, -2.4]$	-6.7% $[-13.3, +0.2]$
Raw messages	-17.5% $[-19.7, -15.7]$	-18.9% $[-21.1, -17.2]$

6.5.2 Optimisation Method

The critic's signal is not specific to evolution strategies. As a comparison, we swap the ES update for GRPO under an otherwise-identical pipeline and repeat the full headline protocol: ten seeds, identical in-training-timeframe selection over the checkpoint grid, and one sealed scoring at 4,096 held-out windows with a norm-matched selection null of its own. Across ten seeds, GRPO shifts WS-21 by -16.0% and Mean-L_1 by -17.3% against the same anchor (its selection null $+1.8\%$, not significant), compared with -17.6% and -17.0% for EGGROLL, so the adversarial reward is optimiser-agnostic. We retain EGGROLL for our framework's pipeline for two reasons. First, GRPO's selection curves are jagged, showing sharp step-to-step swings rather than EGGROLL's smooth rise into the hacking window, which makes results less predictable and harder to analyse. Second, the per-position decomposition of Section 6.6.1 shows contrasting results. GRPO slows accumulation even more than EGGROLL, but buys it with a large level offset: its marginals start 75% above the anchor's in the first 25-message bin, and the slower growth only wins that offset back, leaving its total divergence at the 500-message horizon almost indistinguishable from the anchor (-1.0%) and above EGGROLL's total divergence (-11.3%).

6.6 What Post-Training Changes

The sections above established that a sequence-level objective can be optimised through the matching engine at a thousandth of the pre-training token budget. This

section asks what that optimisation changes beyond the aggregate score: whether it reaches the long-horizon failure that motivated it, how it distributes its gains across features, and what it does to the representation and the forecasting ability the pre-trained model already had. Together the answers mark out the role post-training can play as a second training stage.

6.6.1 Compounding Error

We test the exposure bias premise directly by measuring divergence as a function of continuation position. Over 4,096 held-out context windows on the 28-day test panel, we compare the true continuation against a free-running rollout from the same context, one rollout per window, all rows generated in a single common-random-numbers programme. At each token position we form the empirical marginal across windows over that position's valid sub-alphabet and compute $\mathrm{KL}(\text{true} \,\|\, \text{gen})$, normalised by the entropy of the true marginal to make positions of different difficulty comparable, and averaged over the sampled phases of the new-message block (two near-deterministic phases with vanishing entropy are excluded). Figure 6.2 reports the accumulation view: each curve minus its own first 25-message bin, so every model starts at zero and its endpoint is the excess divergence accumulated over the rollout. Horizon-independent level offsets, including the estimator's finite-sample bias (a real-versus-real control of the same statistic is flat across the rollout), drop out of the comparison.

The pre-trained model's divergence grows 3.1× from the first to the last 25-message bin, direct evidence that its errors compound over free-running generation. Post-training reduces exactly this accumulation, the growth falling to 2.3× and the ten-seed mean accumulating 26% less excess divergence than the anchor over the 500-message horizon (paired day-bootstrap, replicate-wise differences on a draw matrix shared across rows; 95% CI $[-33\%, -20\%]$), with every seed accumulating less than the anchor. The controls locate the source of the reduction: the selection-matched null accumulates 27% more than the anchor (CI $[+15\%, +37\%]$), and the

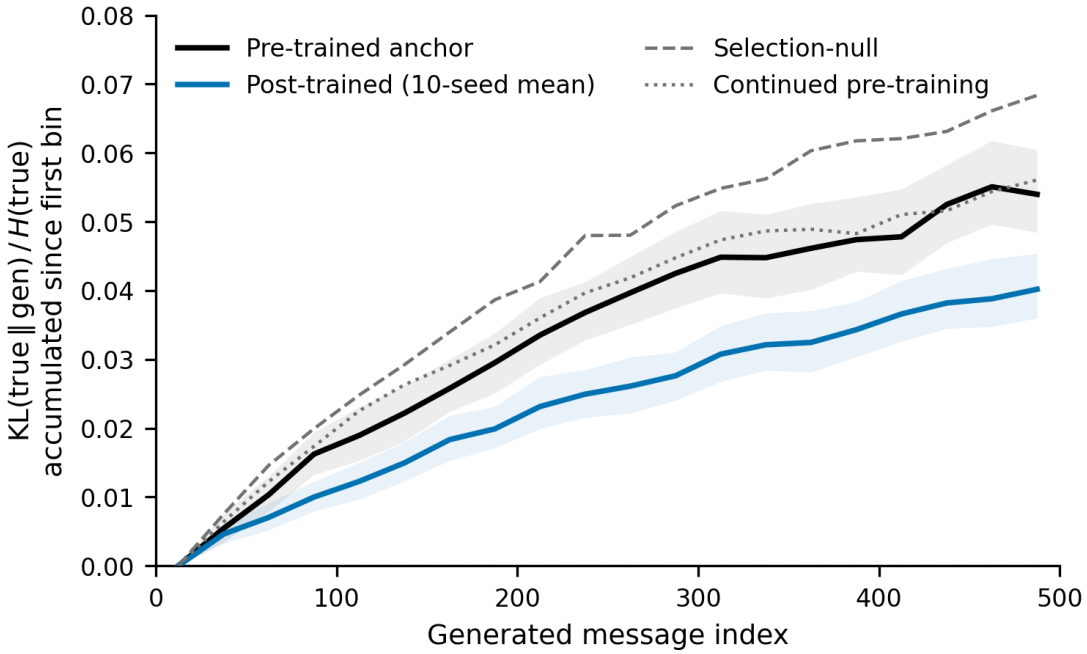


Figure 6.2: Excess divergence, $\mathrm{KL}(\mathrm{true} \,\|\, \mathrm{gen})/H(\mathrm{true})$ accumulated since each curve's first 25-message bin. Post-training accumulates 26% less excess divergence than the anchor, the selection null 27% more, continued pre-training the same. Shaded: 95% day-clustered bootstrap bands.

Continued Pre-Training (CPT) control accumulates at the anchor's rate (+3.9%, CI $[-6\%, +14\%]$). Neither selection pressure nor additional likelihood training slows accumulation; only the critic's sequence-level signal does.

6.6.2 Market Realism

Absolute Value Variations

Beyond the aggregate WS-21, per-feature response follows a regular pattern: the larger a feature's error in the base pre-trained model, the more post-training improves it. Fitting the 10-seed average relative change against the anchor's per-feature Wasserstein level with a saturating power law $y = y_0 + A\,x^p$ across the 21 features gives $R^2 = 0.86$ and Spearman $\rho = -0.76$ (Figure 6.3). The same compression holds across every critic variant of Section 6.5, indicating it is a property of the adversarial signal itself rather than of any particular critic input.

The mechanism follows from the objective rather than from any feature. The critic scores whole rollouts and carries no per-feature term; its discriminative power concentrates wherever the anchor deviates most from the data, and that is where the gradient-free update is steered hardest. As those deviations shrink they stop separating real from generated and the pressure moves elsewhere. This saturating dynamic is why a saturating power law and not a linear trend describes the response. Because nothing in the objective identifies a feature, the pattern is a consequence of adversarial pressure seeking the cheapest separation, not feature-specific tuning.

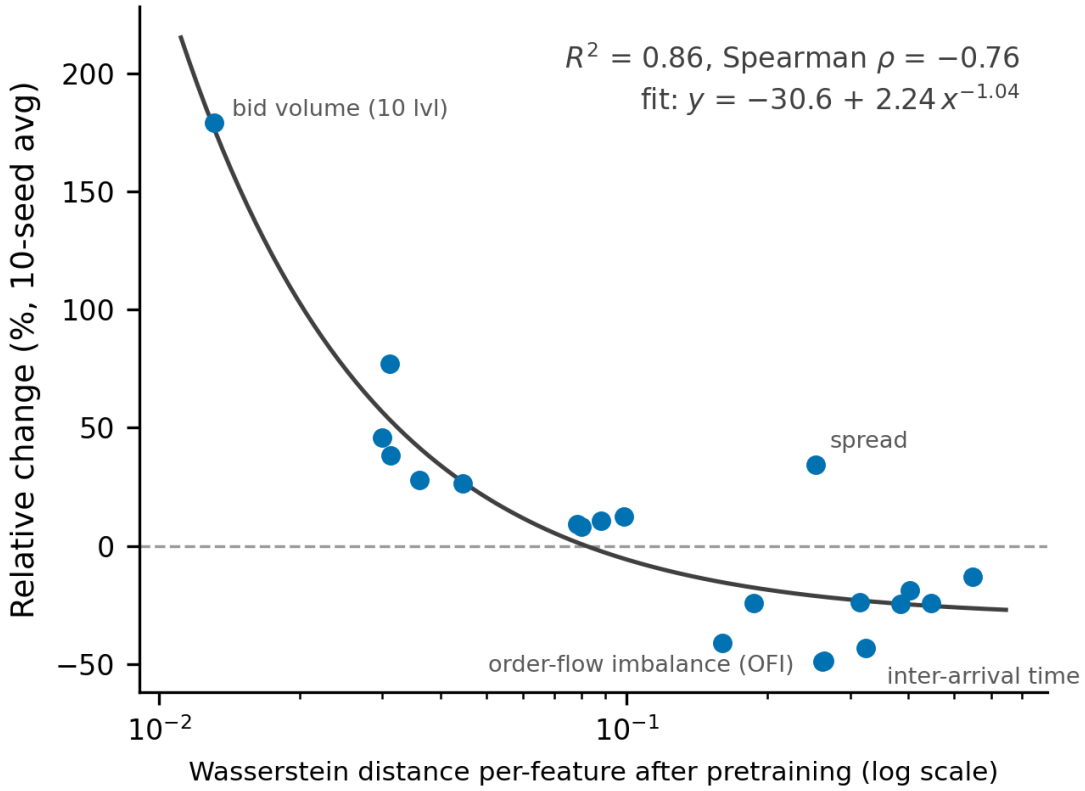


Figure 6.3: Per-feature error shift at test-time, averaged over the 10 post-training seeds. The saturating power-law fit ($R^2 = 0.86$, Spearman $\rho = -0.76$) shows the largest anchor errors improving most; features near the evaluation noise floor (left) swing widely in relative terms, and spread (annotated) is the main outlier.

Two departures are informative. The bid–ask spread, despite carrying only the 9th-largest anchor error, worsens by +34%. Unlike the compressed features it is not a marginal of the message stream but a joint property of the standing book, set by

the interaction of placements and cancellations at the touch: the critic can improve each event type's marginal realism while the book-level equilibrium it induces drifts. It is also self-reinforcing under free running, since a slightly wide spread changes which orders are marketable, which changes the next message, which widens it further. Since the critic scores the message stream but never observes the standing book, this drift is only indirectly visible to the reward; the exchangeable reward module of Section 8.3 offers a potential countermeasure, retargeting a follow-up post-training pass at book-level properties such as the spread itself. Conversely, features near the noise floor worsen only in relative terms: their anchor errors are so small that tiny absolute changes read as large percentages, and their absolute errors remain small after post-training.

Beyond Zero-Sum Shifts

Recent work also characterises the neighbourhood of large pre-trained models as a thicket of directions that improve individual metrics by chance [98], so checkpoint selection over a training run could in principle manufacture apparent gains from undirected drift alone. Here, it could be possible that the generator moves error between statistics, improving whatever the critic currently prices at the expense of what it ignores, without improving general performance. Our selection-null control bounds this channel by passing an equivalent number of random LoRA perturbations through the identical selection pipeline: it ends significantly above the anchor, so selection alone manufactures no gain here, likely because neural thickets are observed at far larger scales (beyond $\sim$1.5 B parameters) than our 78.5 M-parameter generator.

6.6.3 Market Decoding

Chapter 5 established that a pre-trained LOB foundation model carries a usable representation of the market in its recurrent state. Post-training rewrites that model's weights against a reward that never mentions OFI, traded volume or realised volatility, so we ask what the update does to that representation. The

adversarial pressure could degrade the encoded statistics, leave them untouched, or sharpen them if realistic order flow demands a better internal account of that flow. Nothing in the objective decides between these cases, so the question is empirical.

We re-run the probing protocol of Section 5.1 unchanged, on the same held-out GOOG January 2026 panel of 2,014 windows, in three conditions: the frozen pre-trained anchor, the deployed post-trained checkpoint, and a random-weights twin. Two elements differ from Chapter 5. The generator is larger (78.5 M against 8.2 M parameters), and it receives the book as input, so book-derived quantities are partly readable from the input itself and the twin alone reaches $R^2 = 0.65$ to 0.91 on depth, touch and imbalance.

Table 6.3: Best-layer held-out R^2 decoded from the generator's recurrent state before and after adversarial post-training, with a random-weights twin as the floor. Brackets are 95% day-block bootstrap CIs; the two Δ columns give the change under day-grouped k-fold and under forward-chaining CV.

Target (W=500)	Random twin	Pre-trained	Post-trained	Δ	Δ fwd-CV
OFI_1	0.098	0.604 [0.561, 0.658]	0.600 [0.556, 0.653]	−0.005	−0.004
OFI_5	0.068	0.541 [0.437, 0.737]	0.538 [0.434, 0.735]	−0.003	−0.003
Traded volume	0.088	0.599 [0.572, 0.622]	0.596 [0.569, 0.620]	−0.002	−0.005
Realised volatility	0.322	0.661 [0.545, 0.770]	0.634 [0.518, 0.750]	−0.027	+0.021

Post-training leaves market decoding unchanged. No target in Table 6.3 moves outside its confidence interval, under either cross-validation protocol. Widening the comparison to all 40 probes we ran, covering both protocols, the recurrent state and the residual stream, the shorter lookbacks and the book-derived targets, 18 move up and 22 move down, with a median absolute change of 0.003 R^2 and a maximum of 0.039, on log book depth under forward-chaining CV, whose post-training interval still contains the pre-trained value. The equality also holds layer by layer, with the post-trained profile tracing the pre-trained one to two decimal places at all ten layers, so the representation has been neither relocated nor reshaped. This is what the size of the update predicts: an update confined to the interior projection kernels, built from rank-4 LoRA perturbations, held near the anchor by a KL trust region

and costing +0.10% cross-entropy (Section 6.3.2), leaves little room to rewrite what the state encodes.

Absence of an improvement is a limitation, since a stronger representation would have been an argument for post-training beyond distributional realism. Two elements bound what that null means. First, the objective never asked for the improvement: the critic scores the realism of message sequences and carries no term in OFI or volatility, so nothing in the fitness points towards decoding them better. Second, a probe measures what a fixed readout recovers, not what the model uses [36], so equal decodability leaves open how that information is consumed downstream. Improving it would likely take a reward that prices these statistics directly, either by exposing the corresponding features to the critic or by adding explicit terms to the ES fitness, as the exchangeable reward module of Section 8.3 proposes.

6.6.4 Downstream Performance

Our results show that post-training improves market realism and slows error accumulation over free-running rollouts. We therefore ask if that gain also shows in downstream performance or if that improvement is compensated by worse predictive capabilities. We test this on mid-price direction: over 2,048 held-out windows of the same 28-day panel, each model free-runs from a common context and we classify the sign of its mid-price change at horizons of 100 to 1,000 messages against the truth (three classes: up, flat, down). Deltas are paired against the anchor on identical windows, with a day-clustered bootstrap draw matrix shared across rows and horizons.

Table 6.4: Downstream mid-price direction at 2,048 held-out windows (28-day panel). Δ is the mean change in three-class directional accuracy across n=10 seeds relative to the frozen pre-trained anchor, with paired day-clustered bootstrap 95% CIs. Accuracy is preserved at every horizon: no interval separates from zero.

Horizon (messages)	Δ Directional accuracy [95% CI]
100	$+0.011$ [-0.008, +0.031]
250	$+0.012$ [-0.007, +0.028]
500	-0.004 [-0.024, +0.016]
1,000	$+0.009$ [-0.010, +0.027]

Directional accuracy is preserved at every horizon (Table 6.4). This is consistent with the two quantities living on different axes: realism concerns the distribution of order flow, while direction concerns the conditional mean of a near-martingale. Efficient prices leave almost no predictable component to sharpen (the anchor's own correlation with the true move decays to zero within ≈400 messages), so reshaping the flow distribution moves realism without disturbing direction. Preservation is the informative outcome. No term in the critic or the ES reward targets directional prediction, so nothing protects this capability during the update; it is free to degrade, and does not. The compounding-error reduction is therefore not a zero-sum reallocation between capabilities. Rollouts stay closer to the data for longer (Section 6.6.1) while downstream predictive performance carries through intact, and that combination is what makes the refinement usable in practice.

6.7 Summary

This chapter answers the second question of the thesis. A pre-trained generator can be improved by an objective no gradient reaches: a Wasserstein critic trained online on decoded rollouts, optimised with evolutionary search under a KL trust region and selected without the critic, improves held-out realism for every seed at a thousandth of the pre-training token budget, and neither selection alone nor continued pre-training at matched compute reproduces the gain. The ablations fix what the result depends on, from what the critic observes to how its scores become

updates, and the reward is optimiser-agnostic. This result places post-training as a necessary stage that more pre-training does not replace, and an inexpensive one, rather than an all-in-one method. Realism improves (Section 6.6.2), error accumulation slows (Section 6.6.1), all while the model's pre-trained representation and directional accuracy stay intact. Its costs are small and local: +0.10% cross-entropy, and a spread drift attached to the one book-level property the critic never observes. Chapter 7 sets these findings, together with those of Chapter 5, against what integration into trading would require.

7

Discussion

Contents

7.1 Current Understanding of HFT Foundation Models . 82
7.1.1 What the State Encodes 82
7.1.2 Encoding Across Training, Scale and Post-Training . . . 83
7.1.3 Composition of the Pre-Training Corpus 83
7.1.4 Open Questions 84
7.2 Integration into High-Frequency Trading 85
7.2.1 Market Impact 85
7.2.2 Spread, Drift and Tail Events 86

Chapters 5 and 6 each answered one question of this thesis. This chapter reads the two studies together. Section 7.1 consolidates what they establish about foundation models trained on the LOB: what the recurrent state encodes, how the encoding depends on training, scale, post-training and the composition of the corpus, and what remains unexplained. Section 7.2 sets those findings against what integration into a high-frequency trading system would require, and names the gaps that the distributional evaluation of the preceding chapters does not reach.

7.1 Current Understanding of HFT Foundation Models

The experiments of Chapters 5 and 6 characterise foundation models trained on the LOB along three lines: the content of their internal representation, its causal use, and its dependence on training. This section consolidates those findings across the two models studied, the 8.2M message-only model and the 78.5M dual-stream anchor, which differ by an order of magnitude in size and in their inputs and yet support the same account. Trained only to predict the next message, the model has learned not a picture of the market, but its flow.

7.1.1 What the State Encodes

The representation lives in the recurrent state and nowhere else, since the residual stream remembers only the last two to five messages (Figure 5.2), and what it holds is flow rather than a snapshot, OFI decoding five times better from the state than from a readout of the boundary book snapshot while resting depth, which a stale book predicts better, is the one quantity the state summarises only coarsely (Tables 5.1 and 5.2). The state is two things at once. Most of it describes the present, the newest message alone writing more than half of it, but the order-flow signal is drawn from a small long-memory subspace of integrators that the random-weights twin does not have (Figure 5.4). The representation is used, unevenly: steering moves the model's beliefs as each feature's meaning dictates for all three quantities, but once the steered state is rolled out through the engine only the volume push

carries through to the market, the flow push moving the generated order flow but not the price (Table 5.3). It also transfers, the touch and OFI decoding on stocks the model never saw (Figure 5.8).

7.1.2 Encoding Across Training, Scale and Post-Training

Flow representation is the slowest thing the model learns, still rising at the last checkpoint after LOB-Bench realism has stopped moving (Figure 5.7), so a better next-message model carries more of the market in its state than its realism score reveals.

Scale and input change that picture less than one would expect. The 78.5M dual-stream anchor of Chapter 6 is ten times larger than the message-only model and receives the book at every message, yet measured above its own random-weights floor it decodes OFI, traded volume and realised volatility by 0.51, 0.51 and 0.34 of R^2, against 0.58, 0.47 and 0.36 for the 8.2M model (Tables 5.1, 5.2 and 6.3). The comparison is indicative rather than controlled, since the two models were trained on different corpora and the book input raises the twin's floor, but within those limits the flow representation does not grow with parameters. It is set by what the likelihood objective has to learn, and both models have to learn the same thing.

Post-training completes the argument from the other side. Section 6.6.3 found the 78.5M representation unchanged by adversarial post-training, with no target moving outside its interval and the layer profile preserved to two decimal places, so the update that buys 17.6% of realism does so without rewriting what the state encodes. Taken with the training curve, this locates the representation: built by likelihood training, strengthened by more of it, indifferent to parameter count at the scales tested, and not a lever that sequence-level realism pulls on.

7.1.3 Composition of the Pre-Training Corpus

The representation is set by the likelihood objective, and what that objective has to learn is set by the windows it is trained on. A preliminary experiment tests this at

the anchor's scale. A generator of the anchor's size is pre-trained on an interleaved tape, the messages of seven MegaCap-8 stocks merged in time order and each tagged with its stock, and compared at equal compute with the same model pre-trained on single-stock windows. Tape pre-training yields a 20% WS-21 reduction on the mixed stream and the only model with a realistic inter-stock generation pattern when left to run, so the state does hold the cross-section of the market. The same model scores 30% worse WS-21 on a single stock. The composition of the training windows, rather than the content of the corpus, therefore decides what the generator does well. Section 8.3.2 develops the proposal this suggests.

7.1.4 Open Questions

Four observations remain unexplained. Firstly, resting depth is tracked only coarsely, at $R^2 = 0.38$ against 0.64 from a stale book, although the book, and with it its depth, is a deterministic function of the messages the state has read. Secondly, the representation is absent from the middle of the stack for every probe tried, linear and non-linear alike, so how the early layers' code is consumed before it partially reappears at the penultimate layer is not known. Thirdly, the extremes are not represented: OFI decodability falls linearly with the share of variance held by the top 1% of windows, on in-training and unseen stocks alike. Lastly, the volatility axis separates belief from behaviour, since the model expects prices to disperse under a push yet does not generate more volatile paths, a gap between what the state encodes and what the generator does that probing alone cannot resolve.

These limits bear directly on use. A simulator inherits the tails its generator does not represent, an agent trained in it inherits a depth the state only approximates, and a representation read out for forecasting is only as good as the quantities it actually carries. Section 7.2 sets the findings of this chapter against what integration into a trading system would require.

7.2 Integration into High-Frequency Trading

Integrating foundation models into high-frequency trading raises problems that the distributional evaluation of the preceding chapters does not address. Four are examined here: the response of the generated market to an agent's own orders, which no observational score constrains; the spread, which decides the profitability of any strategy that trades inside it; the long-horizon drift of free rollouts; and the tails, of the message stream and of the market it generates, where both the representation and the simulator are weakest.

7.2.1 Market Impact

Every score in this thesis is observational: LOB-Bench, the critic and the position-wise divergence all compare what the model chose to emit with what the market did, and none places an order into the book from outside. Nothing in the training or the evaluation therefore constrains the left-hand side of (2.4), the response of the generated market to an exogenous order, which is its market impact and the quantity an agent trading inside the simulator would meet (Section 2.3.3).

Under that test, impact does emerge in generators of this family trained on the MegaCap-8 corpus: an injected metaorder is answered with an identifiable power law, and a blind control in which the orders fill in the engine but never reach the model attributes the response to the model rather than to book mechanics [99]. The response is not yet the market's. Exponents exceed the square root on every stock tested, 0.73 to 1.21 on GOOG, amplitudes run several times what a real metaorder pays per unit of participation, and no model relaxes after execution, a pattern consistent with persistent order flow that the generated market never compensates.

For integration this fixes the order of work. Distributional realism can be improved at low cost via adversarial post-training, but it does not certify the response an agent would trade against, and the two have to be measured separately. Whether a critic can be trained to price that response, and whether the short

context the evidence implicates can be overcome without longer memory, is untested; Section 8.3.1 takes impact up as a post-training target.

7.2.2 Spread, Drift and Tail Events

Three further properties separate a realistic stream from a tradable market. The first is the bid–ask spread, the one feature that post-training worsens (+34%, Section 6.6.2) for want of any book-level signal in the critic. The spread is where a model's accuracy is converted into profit or loss. Passive strategies, a market maker's quotes or an execution schedule, earn a fraction of it and are mispriced by a simulator that gets it wrong; and a directional strategy built on the mid-price forecasts of Section 5.3 must cross it to trade, so a correct direction is profitable only when the expected move exceeds the half-spread paid on entry and again on exit, a cost that mid-price accuracy alone never accounts for.

The second is drift. A generator that trends when left alone moves the price an agent sees for reasons unrelated to the agent's orders, and the compounding measured in Section 6.6.1, divergence growing 3.1× over 500 messages before post-training and 2.3× after, is the same mechanism read position by position. Post-training slows it and does not remove it, so over horizons of thousands of messages, the scale of a metaorder, the generated market is still moving away from the real one.

The third is the tails, of both the message stream and of the market. In the stream, the rare event types are where validity fails: on the anchor, 0.3% of generated messages cannot be executed at all, a cancellation of an order absent from the book or an execution with no counterparty, against 0.03% for the real stream replayed through the same engine, and a simulator that silently absorbs such orders understates how often a strategy's own orders would be rejected. In the market, the extremes are where the model is weakest (Section 7.1.4), so the bursts of flow and price that decide a strategy's worst days are the episodes a rollout is least able to produce. A profit-and-loss distribution estimated in such a simulator

is therefore narrower than the real one, understating drawdowns, the cost of being caught in a liquidity gap, and the risk limits a live system would breach.

Different applications thus stand at different distances from deployment. As a representation, foundation models are already usable today: a frozen state updated once per message yields market statistics at negligible cost, transfers to unseen instruments, and survives post-training intact. As an environment they are not: the generated market's realism is high and improving, but the response to an agent's orders is too steep, too expensive and does not relax, its spread moves in the wrong direction under the reward that improves everything else, and its tails are the least trusted part of it.

8

Conclusion

This thesis asked two questions of foundation models of order flow: what they come to represent when pre-trained by next-message likelihood, and whether a second training stage, optimised through the matching engine, can improve what they generate. Chapter 5 answered the first by reading and steering the recurrent state, Chapter 6 the second by post-training a generator against an online critic, and Chapter 7 read the two together. We conclude by collecting those findings, their limitations, and setting out the research directions that follow.

8.1 Summary of Findings

Chapter 5 found that a model trained only to predict the next message holds a representation of the market in its recurrent state and nowhere else. What it holds is flow rather than a snapshot, with OFI, traded volume and realised volatility decoding from the state at R^2 of 0.50 to 0.61, far above a random-weights twin and above a single book snapshot, while the residual stream carries almost none of it. The representation is used, since writing a feature's direction into the state moves the model's next-message beliefs as its meaning dictates, though its information

about future prices adds little to that of the visible book. It transfers to stocks never seen in training, and it is the slowest thing the model learns, still improving after LOB-Bench [5] realism has plateaued. With respect to the first question, these findings fix what the model represents: the market as a flow rather than a snapshot, held in the recurrent state and consulted when the next message is predicted.

Chapter 6 showed that such a generator can be improved by an objective no gradient reaches. A Wasserstein critic trained online on losslessly decoded messages scores free rollouts executed through the matching engine, EGGROLL turns those scores into updates built from rank-4 LoRA perturbations under a KL trust region, and checkpoints are selected without the critic. On a sealed held-out panel, post-training lowers WS-21 by 17.6% and Mean-L_1 by 17.0% over ten seeds, each seed's paired interval excluding zero, on a thousandth of the pre-training token budget, while a selection-matched null and continued pre-training at matched compute both end above the anchor. The ablations reveal several training behaviours. Firstly, the reward is eventually hacked, so external selection is part of the method. Secondly, decoding the raw messages roughly doubles the gain of hand-designed descriptors. Thirdly, at a fixed rollout budget the split between directions and contexts is immaterial. Lastly, GRPO under the same critic reaches -16.0%, so the signal is optimiser-agnostic.

Chapter 6 also located what the optimisation changes. It reaches the failure that motivated it, since over 500 generated messages the post-trained seeds accumulate 26% less excess divergence from the true continuation than the anchor, which neither control reproduces. The gains follow the anchor's errors, the largest improving most, with the bid–ask spread the one feature that worsens because the critic never observes the standing book. The representation of Chapter 5 survives the update unchanged and mid-price directional accuracy is preserved at every horizon. Read together in Chapter 7, the two studies place the representation with the likelihood objective rather than with scale or input. Post-training is therefore a corrective stage that more pre-training does not replace: it improves realism and slows error

accumulation at small and local cost, and leaves intact what makes the pre-trained model usable as a representation. What still separates the generated market from a tradable environment is covered in Section 7.2.

8.2 Limitations

Three limitations with respect to the interpretation of our results must be considered to delimit what the findings above establish.

Post-training is selected on the benchmark it is tested on. Checkpoints are chosen by WS-21 over windows from the training period and judged by WS-21 and Mean-L_1 over a later one, so the improvement could in principle be specific to the 21 statistics LOB-Bench measures rather than to realism as such. We mitigated this by asking whether anything the benchmark does not score deteriorated as a consequence. Error accumulation against the true continuation fell, directional accuracy was preserved at every horizon, and market decoding was unchanged, which is evidence against benchmark-specific optimisation without excluding it.

Every score is observational, so the realism established here says nothing about the market impact an agent trading inside the simulator would face, a gap Section 7.2.1 discusses and no experiment here measures.

Neither probing nor steering proves that the model uses what it holds in practice. A probe shows that a quantity can be read from the state by a fixed linear map; it does not show that the model reads it the same way, or at all, when it predicts the next message. Steering addresses this by writing into the state and observing whether the output moves, but the directions we write are linear differences of means, added at one message boundary in every layer at once, so the test can only find a representation that is linear, located where we wrote, and consulted immediately afterwards. A quantity the model uses non-linearly, or later in the stack, would pass the probe and fail the steering test without being unused.

8.3 Future Work

The findings of this thesis direct future research towards pre-training and post-training as a combined pipeline. A sequence-level objective can now be optimised through the matching engine at a thousandth of the pre-training budget, and the representation it acts on is set by pre-training and left intact. Two directions follow: what the reward is pointed at, and what the pre-trained base is.

8.3.1 Retargeting the Reward

Because nothing in the pipeline is specific to aggregate realism, the critic is an exchangeable reward module, and our framework transfers to any property computable from free rollouts, inside or outside the high-frequency literature. Post-training thus positions itself as a midpoint between pre-training and fine-tuning: pre-train once for global market structure, post-train to sharpen a targeted property, and fine-tune the result into a specialised system such as an agentic trading framework or an information-decoding pipeline. The gaps measured in Chapters 6 and 7 name the first targets. The spread worsened for want of a book-level signal in the critic, so a book-level term in the reward is the first candidate. The decoded statistics were left unchanged because nothing rewarded them, so a term in OFI or volatility decodability is the second, and it would test whether a representation can be strengthened without trading realism against it. The most consequential target is market-impact fidelity. Empirical impact obeys tight regularities such as the square-root law [27, 28], and a simulator that reproduces the counterfactual price response to injected orders is the missing prerequisite for training and evaluating trading agents in realistic environments [21, 59]. The paired rollout of Section 2.3.3 yields that response as a scalar (2.6), so the impact profile is an admissible reward. Beyond realism, the same pipeline could post-train a variant of our model for market making, with a reward scoring policies by their expected profit.

The pipeline itself also leaves two questions open. First, all rollouts here span 500 messages, and whether longer training horizons slow error accumulation further

is untested. Second, the trained critic is a realism score in its own right: used in isolation, it could rank generators, flag anomalous order flow, or act as a frozen reward, enabling a controlled study of reward hacking apart from the co-adaptation of Section 6.3.1.

8.3.2 Pre-Training for General Representations

Chapter 5 found that the representation transfers across stocks and is set by the likelihood objective, while Chapters 6 and 7 post-trained a generator pre-trained on a single one. The next step is thus to consider a generator pre-trained on an interleaved tape, the messages of several stocks merged in time order and each tagged with its stock, so that the state holds the cross-section of the market rather than one stock's flow (Figure 8.1). Such a base would make the representation general by construction, providing a foundation that can be specialised to any stock or property via post-training.

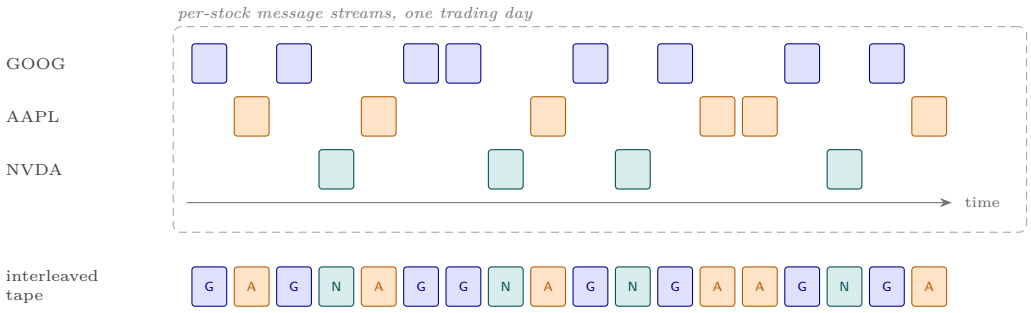


Figure 8.1: The interleaved tape proposal. The message streams of several stocks over one trading day are merged by timestamp into a single stream; each message carries a stock token that the model also predicts.

The preliminary experiment of Section 7.1.3 shows that such a base is within reach: at equal compute, tape pre-training holds the cross-section of the market, and loses fidelity on any single stock. What remains open is whether a single pre-trained base can hold the cross-section of the market and, through post-training, reach the fidelity of a specialist on any one of its stocks. If it can, specialised pre-training would no longer be needed. Such a model could then hold a world representation of the stock market.

The representation itself also leaves questions open. Our probes recover only the concepts we knew to look for; unsupervised tools such as sparse autoencoders could surface features the model has constructed beyond them, and potentially predictors that the market microstructure literature has not yet identified.

Appendices

A

Low-Rank Evolution Strategies

This appendix derives the EGGROLL update of Section 2.6.2 from the low-rank perturbation distribution of (2.13), and states the bound on its error relative to the full-rank Gaussian ES gradient that justifies ranks in the single digits. Both results are due to [12]; they are collected here so that Chapter 2 can state the update and its accuracy without the derivation.

A.1 Derivation of the Update

Sampling from (2.13) rather than from a full-rank Gaussian costs the perturbation distribution its closed-form score, since E is confined to the manifold of rank-r matrices and derivatives of $\log p(E)$ live on its tangent space. The score is therefore replaced by an approximation $\hat{S}(E) \approx \nabla_E \log p(E)$, giving the low-rank update

$$\hat{g}_{\mathrm{LR}} = -\frac{1}{\sigma}\,\mathbb{E}_{E\sim p(E)}\left[\hat{S}(E)\, f(W = M + \sigma E)\right], \tag{A.1}$$

where σ is the perturbation scale and f the fitness. Sending $r \to \infty$ in (2.13)

makes each entry a standardised sum of r independent terms, so the central limit theorem drives $p(E)$ to a matrix Gaussian and returns the Gaussian score $\hat{S}(E) = -E/\sigma_0^4$, which is the $\hat{S}(E) = -E$ used in practice under the normalisation $\sigma_0 = 1$. Substituting it into (A.1), estimating the expectation with a population of N workers, and absorbing $1/\sigma$ and $1/\sigma_0^4$ into the step size leaves the update actually applied, (2.14).

A.2 Error of the Low-Rank Estimator

Writing g_{true} for the full-rank Gaussian ES gradient, the low-rank estimator satisfies $\|\hat{g}_{\text{LR}} - g_{\text{true}}\|_F = O(1/r)$, faster than the $O(1/\sqrt{r})$ that the central limit theorem alone would give [12]. The gain comes from symmetry: an Edgeworth expansion of the density bounds the error by the cumulants of p_0 above second order, and every odd cumulant of a symmetric distribution vanishes, so the leading term is fourth order rather than third. The entries of A and B need only be drawn from a zero-mean, symmetric, absolutely continuous distribution with finite fourth-order moments for the bound to hold; Gaussian entries are one admissible choice. The approximation is already accurate at ranks in the single digits, which is what permits the rank-4 perturbations of Chapter 6.

B

LOB-Bench Scoring Functions

Section 2.3.1 evaluates rollout realism with LOB-Bench [5], whose default battery reduces a stretch of market activity to 21 scalar scores. Table B.1 defines each of them. The aggregates reported throughout Chapter 6, WS-21 and Mean-L_1, are the mean over these 21 scores of the Wasserstein-1 and L_1 distances of (2.3).

Table B.1: The 21 default scoring functions of LOB-Bench [5], referenced from Section 2.3.1. Each maps a sequence of messages and the book states they induce to one number, whose distribution over evaluation windows is compared between real and generated data.

Scoring function	Description
Spread	Best ask minus best bid
Order-book imbalance	(bid size − ask size)/(bid size + ask size) at the touch
Log inter-arrival time	Log time between messages
Log time to cancel	Log time to first cancellation of a limit order
Ask/Bid volume at touch	Volume at the best ask/bid
Ask/Bid volume	Total volume over the top ten levels
Limit ask/bid order depth	Distance from mid-price of a new limit order
Ask/Bid cancellation depth	Distance from mid-price of a cancellation
Limit ask/bid order levels	Price-level index of a new limit order
Ask/Bid cancellation levels	Price-level index of a cancellation
Volume per minute	Traded volume, per-second intervals scaled to a minute
OFI	Order-flow imbalance over a rolling window
OFI (up/stay/down)	OFI conditional on the next mid-price move

C

Hyperparameters

Table C.1 collects every hyperparameter of the two training stages, the critic, the optimisers, the generation loop and the probing protocol, so that each experiment can be reproduced from one place. The choices themselves are motivated where they are made, in Sections 4.3.2, 4.4, 5.1 and 6.1.

Table C.1: Hyperparameters of every experiment in this thesis. Rows spanning both columns apply to both generators.

Quantity	Message-only	Dual-stream
Pre-training (Section 4.3.2)		
Steps	30,180	28,730
Global batch (windows)	160	128
Window length	500 messages, 13,000 tokens	
Muon: 2-D kernels	lr 0.01, weight decay 0.005	
Adam: recurrence parameters	lr 2×10^{-3}	lr 5×10^{-4}
AdamW: embeddings, norms	lr 2×10^{-3}, wd 0.005	lr 5×10^{-4}, wd 0.005
Schedule	1% linear warm-up, cosine to 5% of peak	
Gradient clipping, dropout	unit norm, none	
Post-training (Section 6.1)		
Steps (headline, pilot)	30, 50	
Rollouts per step $G \times Q$	$16 \times 128 = 2{,}048$	
Perturbation rank, σ	$r = 4$, 0.003	
Optimiser, KL trust region	Adam lr 5×10^{-4}, $\lambda = 0.1$	
GRPO variant	G=8 per context, one on-policy step, λ=0.3	
Selection	every 2 steps, WS-21 on 2,048 in-period windows	
Critic (Section 6.1.2)		
Ensemble members K	5 spectral-normalised TCNs behind random projections	
Objective	Wasserstein with R_1 penalty, $\gamma = 10$	
Optimiser	Adam lr 3×10^{-4}, $\beta_1 = 0$	
Input channels	12, median-centred, scaled by 1.4826 MAD	
Real pool	512 windows, redrawn every 5 steps	
Generation (Section 4.4)		
Prefix, continuation	500 real messages, 500 generated	
Sampling	top-50 admissible tokens, proportional to score	
Book initialisation	10 levels a side	500 levels a side
Probing (Section 5.1)		
Probe	kernel ridge, layer and λ by nested inner CV	
Cross-validation	day-grouped 5-fold, and forward chaining	
Panel	2,014 windows, GOOG, January 2026	
Intervals	95% day-block bootstrap	
Steering dose	$\alpha = 1$ is one tercile displacement	

References

[1] Peer Nagy et al. 'Generative AI for End-to-End Limit Order Book Modelling: A Token-Level Autoregressive Generative Model of Message Flow Using a Deep State Space Network'. In: *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF '23)*. ACM, 2023, pp. 91–99. DOI: 10.1145/3604237.3626898.

[2] Aaron Wheeler and Jeffrey D. Varner. *MarketGPT: Developing a Pre-trained transformer (GPT) for Modeling Financial Time Series*. 2024. arXiv: 2411.16585.

[3] Junjie Li et al. 'MarS: a Financial Market Simulation Engine Powered by Generative Foundation Model'. In: *The Thirteenth International Conference on Learning Representations (ICLR)*. 2025.

[4] Ruihong Huang and Tomas Polak. *LOBSTER: Limit Order Book Reconstruction System*. Technical Report. Available at SSRN 1977207. 2011.

[5] Peer Nagy et al. 'LOB-Bench: Benchmarking Generative AI for Finance – an Application to Limit Order Book Data'. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. Vol. 267. PMLR, 2025, pp. 45437–45460.

[6] Kenneth Li et al. 'Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task'. In: *The Eleventh International Conference on Learning Representations*. 2023.

[7] Neel Nanda, Andrew Lee and Martin Wattenberg. 'Emergent Linear Representations in World Models of Self-Supervised Sequence Models'. In: *Proc. of the 6th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. 2023, pp. 16–30.

[8] Adam Karvonen. 'Emergent World Models and Latent Variable Estimation in Chess-Playing Language Models'. In: *First Conference on Language Modeling (COLM)*. 2024.

[9] Aramis Marti-Shahandeh et al. 'Recurrent Sequence Models Outperform Transformers in Closed-Loop Limit Order Book Generation'. Manuscript, University of Oxford. Under review. 2026.

[10] Samy Bengio et al. 'Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks'. In: *Advances in Neural Information Processing Systems 28*. 2015.

[11] Marc'Aurelio Ranzato et al. 'Sequence Level Training with Recurrent Neural Networks'. In: *International Conference on Learning Representations*. 2016.

[12] Bidipta Sarkar et al. 'Evolution Strategies at the Hyperscale'. In: *arXiv preprint arXiv:2511.16652* (2025).

[13] Zhihong Shao et al. 'DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models'. In: *arXiv preprint arXiv:2402.03300* (2024).

[14] Junyi Li et al. 'Generating Realistic Stock Market Order Streams'. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 01. 2020, pp. 727–734. DOI: 10.1609/aaai.v34i01.5415.

[15] Andrea Coletta et al. 'Learning to Simulate Realistic Limit Order Book Markets from Data as a World Agent'. In: *Proceedings of the Third ACM International Conference on AI in Finance (ICAIF '22)*. ACM, 2022, pp. 428–436. DOI: 10.1145/3533271.3561753.

[16] Leonardo Berti, Bardh Prenkaj and Paola Velardi. 'TRADES: Generating Realistic Market Simulations with Diffusion Models'. In: *ECAI 2025 – 28th European Conference on Artificial Intelligence*. IOS Press, 2025, pp. 3703–3710. DOI: 10.3233/FAIA251249.

[17] Martin D Gould et al. 'Limit order books'. In: *Quantitative Finance* 13.11 (2013), pp. 1709–1742.

[18] Jean-Philippe Bouchaud et al. *Trades, quotes and prices: financial markets under the microscope*. Cambridge University Press, 2018.

[19] Rama Cont, Arseniy Kukanov and Sasha Stoikov. 'The Price Impact of Order Book Events'. In: *Journal of Financial Econometrics* 12.1 (2014), pp. 47–88.

[20] Petter N Kolm, Jeremy Turiel and Nicholas Westray. 'Deep Order Flow Imbalance: Extracting Alpha at Multiple Horizons from the Limit Order Book'. In: *Mathematical Finance* 33.4 (2023). SSRN 3900141, pp. 1044–1081.

[21] Sascha Frey et al. 'JAX-LOB: A GPU-Accelerated Limit Order Book Simulator to Unlock Large Scale Reinforcement Learning for Trading'. In: *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF '23)*. ACM, 2023, pp. 583–591. DOI: 10.1145/3604237.3626880.

[22] Aakash Lahoti et al. 'Mamba-3: Improved sequence modeling using state space principles'. In: *The Fourteenth International Conference on Learning Representations (ICLR)*. arXiv:2603.15569. 2026.

[23] Tri Dao and Albert Gu. 'Transformers are SSMs: generalized models and efficient algorithms through structured state space duality'. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. PMLR, 2024, pp. 10041–10071.

[24] Jared Kaplan et al. 'Scaling laws for neural language models'. In: *arXiv preprint arXiv:2001.08361* (2020).

[25] Jordan Hoffmann et al. 'Training compute-optimal large language models'. In: *arXiv preprint arXiv:2203.15556* (2022).

[26] Kang Li et al. 'Scaling Laws for Order Flow Generation'. Under review at ICLR. 2027.

[27] Jean-Philippe Bouchaud, J. Doyne Farmer and Fabrizio Lillo. 'How Markets Slowly Digest Changes in Supply and Demand'. In: *Handbook of Financial Markets: Dynamics and Evolution*. Elsevier, 2009, pp. 57–160.

[28] Robert Almgren et al. 'Direct Estimation of Equity Market Impact'. In: *Risk* 18.7 (2005), pp. 58–62.

[29] Bence Tóth et al. 'Anomalous Price Impact and the Critical Nature of Liquidity in Financial Markets'. In: *Physical Review X* 1.2 (2011), p. 021006.

[30] Yuki Sato and Kiyoshi Kanazawa. 'Strict Universality of the Square-Root Law in Price Impact Across Stocks: A Complete Survey of the Tokyo Stock Exchange'. In: *Physical Review Letters* 135 (2025), p. 257401.

[31] Jean-Philippe Bouchaud et al. 'Fluctuations and Response in Financial Markets: The Subtle Nature of 'Random' Price Changes'. In: *Quantitative Finance* 4.2 (2004), pp. 176–190.

[32] Esteban Moro et al. 'Market Impact and Trading Profile of Hidden Orders in Stock Markets'. In: *Physical Review E* 80.6 (2009), p. 066102.

[33] Frédéric Bucci et al. 'Slow Decay of Impact in Equity Markets: Insights from the ANcerno Database'. In: *Market Microstructure and Liquidity* 4.03n04 (2019), p. 1950006.

[34] Guillaume Maitrier et al. 'The "Double" Square-Root Law: Evidence for the Mechanical Origin of Market Impact Using Tokyo Stock Exchange Data'. In: *arXiv preprint arXiv:2502.16246* (2025).

[35] Ashkan Golgoon, Khashayar Filom and Arjun Ravi Kannan. 'Mechanistic interpretability of large language models with applications to the financial services industry'. In: *Proc. 5th ACM Int. Conf. on AI in Finance*. 2024, pp. 660–668.

[36] Yonatan Belinkov. 'Probing classifiers: Promises, shortcomings, and advances'. In: *Computational Linguistics* 48.1 (2022), pp. 207–219.

[37] Alexander Matt Turner et al. *Steering Language Models With Activation Engineering*. 2023. arXiv: `2308.10248 [cs.CL]`.

[38] Nina Rimsky et al. 'Steering Llama 2 via Contrastive Activation Addition'. In: *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Aug. 2024, pp. 15504–15522.

[39] Ian J. Goodfellow et al. 'Generative Adversarial Nets'. In: *Advances in Neural Information Processing Systems 27*. 2014.

[40] Martin Arjovsky, Soumith Chintala and Léon Bottou. 'Wasserstein Generative Adversarial Networks'. In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. 2017, pp. 214–223.

[41] Ishaan Gulrajani et al. 'Improved Training of Wasserstein GANs'. In: *Advances in Neural Information Processing Systems 30*. 2017.

[42] Takeru Miyato et al. 'Spectral Normalization for Generative Adversarial Networks'. In: *International Conference on Learning Representations*. 2018.

[43] Martin Heusel et al. 'GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium'. In: *Advances in Neural Information Processing Systems 30*. 2017.

[44] Lars Mescheder, Andreas Geiger and Sebastian Nowozin. 'Which Training Methods for GANs do actually Converge?' In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. 2018, pp. 3481–3490.

[45] James C. Spall. 'Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation'. In: *IEEE Transactions on Automatic Control* 37.3 (1992), pp. 332–341.

[46] Daan Wierstra et al. 'Natural Evolution Strategies'. In: *Journal of Machine Learning Research* 15.27 (2014), pp. 949–980.

[47] Ronald J. Williams. 'Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning'. In: *Machine Learning* 8 (1992), pp. 229–256.

[48] John Schulman et al. 'Proximal Policy Optimization Algorithms'. In: *arXiv preprint arXiv:1707.06347* (2017).

[49] John Schulman et al. 'Trust Region Policy Optimization'. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. 2015, pp. 1889–1897.

[50] Lantao Yu et al. 'SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient'. In: *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*. 2017, pp. 2852–2858.

[51] Tim Salimans et al. 'Evolution Strategies as a Scalable Alternative to Reinforcement Learning'. In: *arXiv preprint arXiv:1703.03864* (2017).

[52] Edward J. Hu et al. 'LoRA: Low-Rank Adaptation of Large Language Models'. In: *International Conference on Learning Representations*. 2022.

[53] Rama Cont, Mihai Cucuringu and Chao Zhang. 'Cross-Impact of Order Flow Imbalance in Equity Markets'. In: *Quantitative Finance* 23.10 (2023), pp. 1373–1393.

[54] Zihao Zhang, Stefan Zohren and Stephen Roberts. 'DeepLOB: Deep convolutional neural networks for limit order books'. In: *IEEE Transactions on Signal Processing* 67.11 (2019), pp. 3001–3012.

[55] James Wallbridge. 'Transformers for limit order books'. In: *arXiv preprint arXiv:2003.00130* (2020).

[56] Justin Sirignano and Rama Cont. 'Universal Features of Price Formation in Financial Markets: Perspectives from Deep Learning'. In: *Quantitative Finance* 19.9 (2019). arXiv:1803.06917, pp. 1449–1459.

[57] Matteo Prata et al. 'LOB-based deep learning models for stock price trend prediction: a benchmark study'. In: *Artificial Intelligence Review* 57.5 (2024), p. 116.

[58] Yang Li and Zhi Chen. 'ByteGen: A Tokenizer-Free Generative Model for Orderbook Events in Byte Space'. In: *arXiv preprint arXiv:2508.02247* (2025).

[59] David Byrd, Maria Hybinette and Tucker Hybinette Balch. 'ABIDES: Towards High-Fidelity Multi-Agent Market Simulation'. In: *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS '20)*. ACM, 2020, pp. 11–22. DOI: 10.1145/3384441.3395986.

[60] Andrea Coletta et al. 'Towards Realistic Market Simulations: A Generative Adversarial Networks Approach'. In: *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF '21)*. ACM, 2021, pp. 1–9. DOI: 10.1145/3490354.3494411.

[61] Andrea Coletta et al. 'Conditional Generators for Limit Order Book Environments: Explainability, Challenges, and Robustness'. In: *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF '23)*. ACM, 2023, pp. 27–35. DOI: 10.1145/3604237.3626854.

[62] Hanna Hultin et al. 'A Generative Model of a Limit Order Book Using Recurrent Neural Networks'. In: *Quantitative Finance* 23.6 (2023), pp. 931–958. DOI: 10.1080/14697688.2023.2205583.

[63] Rama Cont et al. *Limit order book simulation with generative adversarial networks*. SSRN 4512356. 2023.

[64] Alfred Backhouse et al. *Painting the market: generative diffusion models for financial limit order book simulation and forecasting*. 2025. arXiv: 2509.05107 [q-fin.TR].

[65] Zhuohan Wang and Carmine Ventre. 'DiffLOB: Diffusion Models for Counterfactual Generation in Limit Order Books'. In: *arXiv:2602.03776* (2026).

[66] Eljas Linna et al. 'LOBERT: Generative AI Foundation Model for Limit Order Book Messages'. In: *arXiv preprint arXiv:2511.12563* (2025).

[67] Maxime Kawawa-Beaudan et al. *TradeFM: A Generative Foundation Model for Trade-flow and Market Microstructure*. 2026. arXiv: 2602.23784 [cs.LG].

[68] Charles Jin and Martin Rinard. 'Emergent Representations of Program Semantics in Language Models Trained on Programs'. In: *International Conference on Machine Learning (ICML)*. arXiv:2305.11169. 2024.

[69] Alexander Rives et al. 'Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences'. In: *Proceedings of the National Academy of Sciences* 118.15 (2021), e2016239118.

[70] Wes Gurnee and Max Tegmark. 'Language Models Represent Space and Time'. In: *The Twelfth International Conference on Learning Representations*. 2024.

[71] Jennifer Lin et al. *OthelloGPT Learned a Bag of Heuristics*. Alignment Forum / LessWrong. 2024.

[72] Keyon Vafa et al. 'Evaluating the world model implicit in a generative model'. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 26941–26975.

[73] Keyon Vafa et al. 'What Has a Foundation Model Found? Using Inductive Bias to Probe for World Models'. In: *Forty-second International Conference on Machine Learning*. 2025.

[74] András Balogh and Márk Jelasity. 'Verification of the Implicit World Model in a Generative Model via Adversarial Sequences'. In: *The Fourteenth International Conference on Learning Representations*. arXiv:2602.05903. 2026.

[75] Ameen Ali Ali, Itamar Zimerman and Lior Wolf. 'The Hidden Attention of Mamba Models'. In: *Proc. of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. arXiv:2403.01590. 2025, pp. 1516–1534.

[76] Hugo Pitorro and Marcos Vinicius Treviso. 'LaTIM: Measuring latent token-to-token interactions in Mamba models'. In: *Proc. of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 24478–24493.

[77] Farnoush Rezaei Jafari et al. 'MambaLRP: Explaining selective state space sequence models'. In: *Advances in neural information processing systems* 37 (2024), pp. 118540–118570.

[78] Arnab Sen Sharma, David Atkinson and David Bau. 'Locating and Editing Factual Associations in Mamba'. In: *First Conference on Language Modeling (COLM)*. arXiv:2404.03646. 2024.

[79] Junxuan Wang et al. 'Towards Universality: Studying Mechanistic Similarity Across Language Model Architectures'. In: *The Thirteenth International Conference on Learning Representations (ICLR)*. arXiv:2410.06672. 2025.

[80] Gonçalo Paulo, Thomas Marshall and Nora Belrose. 'Does Transformer Interpretability Transfer to RNNs?' In: *arXiv preprint arXiv:2404.05971* (2024).

[81] Jack Young. 'WriteSAE: Sparse Autoencoders for Recurrent State'. In: *arXiv preprint arXiv:2605.12770* (2026).

[82] Jiwon Jung and Kiseop Lee. 'Attention-based reading, highlighting, and forecasting of the limit order book'. In: *Quantitative Finance* 25.7 (2025), pp. 1015–1027.

[83] Hariom Tatsat and Ariye Shater. 'Beyond the black box: Interpretability of LLMs in finance'. In: *arXiv preprint arXiv:2505.24650* (2025).

[84] Marco Molinari et al. 'Interpretable Company Similarity with Sparse Autoencoders'. In: *Proc. 63rd Annu. Meeting of the Assoc. for Comput. Linguistics (Volume 6: Industry Track)*. 2025, pp. 1027–1038.

[85] Hui Chen et al. *A Financial Brain Scan of the LLM*. 2025. arXiv: 2508.21285.

[86] Michał Wiliński et al. 'Exploring Representations and Interventions in Time Series Foundation Models'. In: *Forty-second International Conference on Machine Learning*. 2025.

[87] Debdeep Sanyal et al. 'time2time: Causal Intervention in Hidden States to Simulate Rare Events in Time Series Foundation Models'. In: *NeurIPS Workshop on Recent Advances in Time Series Foundation Models (BERT2S)*. arXiv:2509.05801. 2025.

[88] Xiaoyu Huang and Lulu Wang. 'Emergent Latent-State Computation under Stochastic Volatility'. In: *arXiv:2607.25459* (2026).

[89] Shanchuan Lin et al. 'Diffusion Adversarial Post-Training for One-Step Video Generation'. In: *arXiv preprint arXiv:2501.08316* (2025). ICML 2025.

[90] Aaron Grattafiori et al. 'The Llama 3 Herd of Models'. In: *arXiv:2407.21783* (2024).

[91] Leo Gao, John Schulman and Jacob Hilton. 'Scaling Laws for Reward Model Overoptimization'. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. 2023, pp. 10835–10866.

[92] Xin Qiu et al. 'Evolution Strategies at Scale: LLM Fine-Tuning Beyond Reinforcement Learning'. In: *arXiv preprint arXiv:2509.24372* (2025).

[93] Keller Jordan et al. *Muon: An optimizer for hidden layers in neural networks*. 2024. URL: https://kellerjordan.github.io/posts/muon/.

[94] Diederik P. Kingma and Jimmy Ba. 'Adam: A Method for Stochastic Optimization'. In: *International Conference on Learning Representations*. 2015.

[95] Ilya Loshchilov and Frank Hutter. 'Decoupled Weight Decay Regularization'. In: *International Conference on Learning Representations*. 2019.

[96] Simon McIntosh-Smith, Sadaf R. Alam and Christopher Woods. 'Isambard-AI: a leadership class supercomputer optimised specifically for Artificial Intelligence'. In: *arXiv preprint arXiv:2410.11199* (2024).

[97] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. 2018. URL: `http://github.com/jax-ml/jax`.

[98] Yulu Gan and Phillip Isola. 'Neural Thickets: Diverse Task Experts Are Dense Around Pretrained Weights'. In: *arXiv preprint arXiv:2603.12228* (2026).

[99] George Nigmatulin et al. 'LOB-Impact Benchmark: Emergent Market Impact in Generative Limit Order Book Models'. Manuscript, University of Oxford. Under review. 2026.